
Análisis de la arquitectura neuronal de los
transformadores
Neural architecture analysis of transformers



Trabajo de Fin de Grado
Curso 2025–2026

Autor

Adrian Carlos Skaczylo Sawicka

Director

Miguel Palomino Tarjuelo

Doble Grado en Matemáticas e Ingeniería Informática
Facultad de Matemáticas
Universidad Complutense de Madrid

Análisis de la arquitectura neuronal de los
transformadores
*Neural architecture analysis of
transformers*

Trabajo de Fin de Grado en Matemáticas e Ingeniería
Informática

Autor

Adrian Carlos Skaczylo Sawicka

Director

Miguel Palomino Tarjuelo

Colaborador

Colaborador no definido. Usa \colaboradorPortada

Convocatoria: *Febrero/Junio/Septiembre 2026*

Doble Grado en Matemáticas e Ingeniería Informática
Facultad de Matemáticas
Universidad Complutense de Madrid

DIA de MES de AÑO

Resumen

Análisis de la arquitectura neuronal de los transformadores

El transformador (*transformer*) es una arquitectura utilizada en las redes neurales que apareció en 2017 y ha revolucionado el campo del procesamiento de lenguaje natural (con los modelos extensos de lenguaje), además de generar resultados excelentes en otras áreas como la visión por computadora o en tareas que requieren la combinación de múltiples tipos de datos (imágenes, texto, audio, vídeo). Se basan en un concepto de procesamiento conocido como “atención” que permite a la red asignar diferentes pesos a entradas distintas.

En este trabajo se pretende estudiar esta arquitectura y presentarla de manera didáctica, analizando cuáles son las intuiciones que la motivan y las justificaciones matemáticas, hasta donde las haya, que la hacen posible. También se realizarán implementaciones cuya naturaleza puede variar conforme se vaya realizando el trabajo, pero que en principio estarían orientadas a ilustrar cómo cambios en la arquitectura y/o en el valor de los pesos pueden afectar el rendimiento del modelo.

Abstract

Neural architecture analysis of transformers

The transformer is an architecture used in neural networks that emerged in 2017 and has revolutionized the field of natural language processing (with large language models), as well as producing excellent results in other areas such as computer vision and in tasks that require combining multiple types of data (images, text, audio, video). It is based on a processing concept known as “attention”, which allows the network to assign different weights to different inputs.

This work aims to study this architecture and present it in a didactic manner, analyzing the intuitions that motivate it and the mathematical justifications, where applicable, that make it possible. Implementations will also be developed, whose nature may vary as the work progresses, but which will initially be oriented toward illustrating how changes in the architecture and/or in the values of the weights can affect the model’s performance

Índice

1. Introducción	1
1.1. De los primeros pasos a la revolución del transformador	1
1.2. Objetivos de este trabajo	3
2. Redes Neuronales	5
2.1. Redes Neuronales	5
2.2. Softmax	7
2.3. Normalización de Capa (<i>LayerNorm</i>)	7
3. Transformador	9
3.1. Procesamiento	9
3.2. Coeficientes de atención	11
3.3. Consulta, Clave y Valor	12
3.4. Atención Múltiple	13
3.5. Transformador: Arquitectura	15
4. Lenguaje Natural	17
4.1. Tokenización	17
4.2. <i>Embeddings</i>	19
4.3. Transformadores	21
4.3.1. Decodificador	22
4.3.2. Codificador	26
4.3.3. Codificador-Decodificador	29
5. Implementación	31
5.1. Conjunto de datos: OPUS100	32
5.1.1. Filtrado y calidad de los datos	32
5.1.2. Estructura de los datos	33
5.2. Vocabulario	36
5.3. Transformador	37
5.3.1. Codificación posicional (<i>Positional Encoding</i>)	37
5.3.2. Hiperparámetros	39

5.3.3.	Entrenamiento	39
5.3.4.	Planificación de la tasa de aprendizaje	40
5.3.5.	Metodología de trabajo y experiencia personal	42
5.3.6.	Resultados	43
5.4.	Detalles de implementación	45
5.4.1.	Escalado del producto escalar	46
5.4.2.	Paralelismo y relleno (<i>padding</i>)	47

Bibliografía	51
---------------------	-----------

Introducción

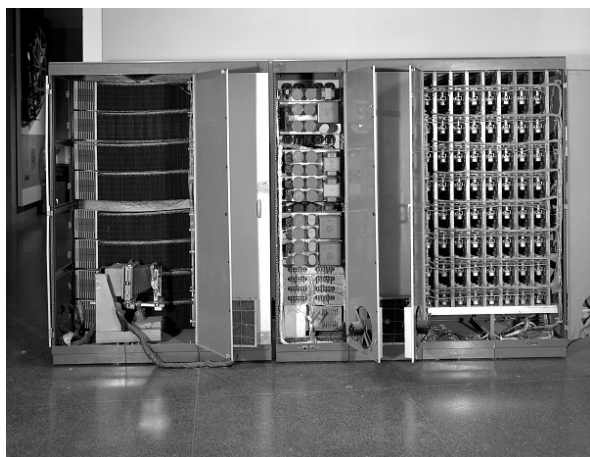
1.1. De los primeros pasos a la revolución del transformador

En 1950, Alan Turing publicó un trabajo fundacional que cambió para siempre cómo pensamos sobre máquinas inteligentes. Propuso el Test de Turing: una prueba donde un interrogador humano conversa por escrito con dos entidades sin saber cuál es persona y cuál máquina. Si la máquina logra ser indistinguible de un humano, ha demostrado verdadera inteligencia. La genialidad de esta idea no estaba en medir cuánto sabía la máquina, sino en evaluar si podía pensar y comunicarse de forma similar a nosotros. Pero Turing fue mucho más allá: en ese mismo trabajo introdujo conceptos que parecían ciencia ficción en su época: aprendizaje automático, donde máquinas mejoran con la experiencia; algoritmos genéticos, inspirados en evolución natural; y aprendizaje por refuerzo, donde máquinas aprenden por ensayo y error. Estas ideas sembraron una revolución que aún hoy continúa.

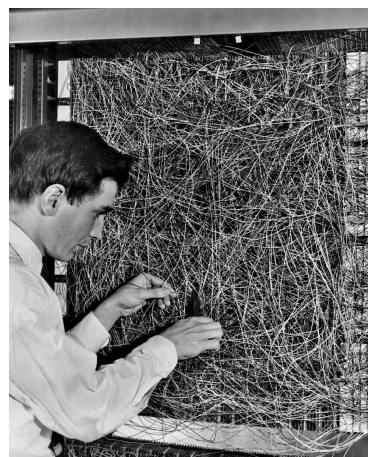
Mientras Turing escribía estas visiones futuristas, otros investigadores ya exploraban caminos paralelos. En 1943, Warren McCulloch y Walter Pitts publicaron un trabajo pionero que formalizó matemáticamente cómo funcionaba una neurona artificial. Inspirándose en la biología del cerebro pero usando lógica rigurosa, propusieron el primer modelo formal que permitía a máquinas procesar información y aprender, no mediante reglas explícitas sino a través de patrones en los datos. Esta fue la chispa que encendería el fuego de las redes neuronales.

Seis años después de Turing, en 1956, sucedió algo crucial: investigadores como John McCarthy, Marvin Minsky, Allen Newell y Herbert Simon se reunieron en una conferencia en Dartmouth College que marcaría el nacimiento oficial de la Inteligencia Artificial como disciplina formal. El optimismo era contagioso. Estos investigadores creían que cualquier aspecto de la inteligencia humana podía ser descrito con precisión matemática y, por tanto, simulado en una máquina. Algunos predijeron audazmente que en apenas diez años tendríamos máquinas tan inteligentes como los humanos.

Mientras unos perseguían razonamiento lógico puro, otros exploraban un enfoque radicalmente diferente: redes neuronales que pudieran aprender del mundo. Frank



(a) Mark I Perceptron en el Laboratorio Aeronáutico de Cornell



(b) Frank Rosenblatt ajustando el cableado del Perceptron.

Figura 1.1: A la izquierda, el perceptron creado por Frank Rosenblatt llamado *Mark I Perceptron*. A la derecha, Frank Rosenblatt trabajando en las conexiones del perceptrón.

Rosenblatt presentó el perceptrón en 1958, una máquina que demostraban que era posible construir algoritmos que mejoraban con la práctica, tal como lo hacemos los humanos. Los periódicos hablaban del futuro prometedor de estas máquinas pensantes; la publicidad hablaba de máquinas que caminarían, hablarían y serían conscientes de su existencia. El entusiasmo era casi ilimitado.

Pero la realidad es más compleja que las predicciones. En 1969, Marvin Minsky y Seymour Papert publicaron un análisis matemático devastador demostrando que los perceptrones simples tenían limitaciones fundamentales insuperables: había problemas que simplemente no podían resolver. Este descubrimiento fue como una ducha de agua fría. La financiación se redujo drásticamente, los periódicos perdieron interés y muchos investigadores abandonaron el campo.

Afortunadamente, el resurgimiento llegó en los años 1980s. Investigadores de múltiples grupos, trabajando de forma independiente, redescubrieron el algoritmo de retropropagación, una técnica elegante para entrenar redes neuronales profundas con múltiples capas. De repente, era posible entrenar redes mucho más complejas.

Durante los años siguientes, especialmente en los 2000s y 2010s, el campo experimentó una evolución espectacular impulsado por tres factores: la disponibilidad de enormes volúmenes de datos en internet, el aumento de la capacidad computacional (especialmente el uso de unidades gráficas o *GPUs*), y algoritmos más sofisticados. Esto dio lugar a la era del aprendizaje profundo (*deep learning*), donde redes neuronales con muchas capas lograron hazañas extraordinarias en reconocimiento de imágenes y juegos complejos. Sin embargo, cuando se trataba de procesar lenguaje y secuencias de texto, la mejor solución seguía siendo las redes neuronales recurrentes (*RNNs*), que procesaban información palabra por palabra. Pero estas redes tenían un talón de Aquiles: la información crucial se perdía conforme procesaban secuencias más largas y sin capacidad de paralelismo, lo que resultaba en sistemas lentos e ineficientes.

En 2017, un equipo de investigadores de Google publicó un trabajo titulado “*Attention Is All You Need*” que introdujo el transformador: una arquitectura basada en un mecanismo de autoatención (*self-attention*) que permitía procesar todas las palabras de una secuencia simultáneamente en lugar de una a una. Aunque parecía un cambio técnico menor, sus consecuencias fueron revolucionarias.

De esta arquitectura brotaron los modelos extensos de lenguaje (*Large Language Models*, LLMs): GPT, BERT, Gemini y otros. Estos sistemas, entrenados con cantidades masivas de texto de internet, desarrollaron capacidades que parecían imposibles años atrás. Pueden escribir ensayos convincentes, traducir entre idiomas preservando significado, generar código de programación, y explicar conceptos complejos de formas que un estudiante puede entender. El salto cualitativo en estas capacidades ha sido tan alto que ha reconfigurado completamente campos como la traducción automática, el reconocimiento de voz, el análisis médico y la investigación científica. Pero la revolución va mucho más allá de lo técnico, pues estos modelos están presentes en nuestra vida cotidiana y su mayor logro es su asombrosa facilidad de uso, ya que al interactuar directamente mediante un lenguaje natural han derribado las barreras digitales tradicionales, permitiendo así que cualquier persona, incluso personas mayores o personas completamente ajenas a la tecnología, los utilicen a diario de forma intuitiva. Al facilitar el acceso a estas herramientas, no solo han transformado cómo escribimos o cómo trabajamos sino también cómo interactuamos con el conocimiento.

Sin embargo, a pesar de que miles de millones de personas usan estas tecnologías diariamente, relativamente pocos entienden realmente cómo funcionan. La mayoría interactúa con estos sistemas como si fueran cajas mágicas, sin comprender los principios matemáticos que subyacen a su funcionamiento y, lo peor de todo, que toman estos modelos como fuentes fiables de información.

1.2. Objetivos de este trabajo

El objetivo central de este trabajo es construir paso a paso una comprensión integral de cómo funcionan los transformadores, de tal forma que quien lo lea pueda apreciar los principios matemáticos y conceptuales que subyacen a estas tecnologías.

Para lograrlo, en primer lugar se estudiará la fundamentación teórica de las redes neuronales, estableciendo los cimientos conceptuales necesarios para comprender arquitecturas más sofisticadas. Posteriormente, se analizará en profundidad la arquitectura del transformador y su mecanismo central de autoatención, explicando de qué forma esta innovación supera las limitaciones críticas de los modelos recurrentes clásicos y revoluciona el procesamiento de secuencias al permitir el paralelismo computacional.

A continuación, se explorará cómo los transformadores transformaron el procesamiento del lenguaje natural que originaron los modelos extensos de lenguaje (LLMs), estudiando los procesos clave que hacen posible que estos sistemas generen lenguaje coherente y contextualmente apropiado. Finalmente, se implementará un transformador tipo codificador-decodificador para traducción automática, demostrando que los principios teóricos explicados poseen validez práctica cuando se aplican a pro-

blemas tangibles del mundo real.

Capítulo 2

Redes Neuronales

Este capítulo tiene como objetivo establecer los cimientos técnicos y la notación que guiarán el resto del trabajo. Comenzaremos formalizando la notación básica de las **redes neuronales**, desde sus estructuras más simples hasta las arquitecturas profundas, para comprender cómo los datos se transforman. Posteriormente, definiremos varias funciones que nos permitirán entender la arquitectura del transformador en los siguientes capítulos.

2.1. Redes Neuronales

Consideremos una red neuronal que mapea un vector de entrada $\mathbf{x} \in \mathbb{R}^{D_i}$ a un vector de salida $\mathbf{y} \in \mathbb{R}^{D_o}$, expresada como $\mathbf{y} = f[\mathbf{x}, \phi]$, donde ϕ representa el conjunto de parámetros del modelo¹. La arquitectura se define por la presencia de K capas ocultas, cada una de ellas con una dimensionalidad D_k para $k = 1, \dots, K$ que representa el número de unidades ocultas o neuronas.

Los elementos que componen la red son:

- **Vectores de activación:** $\mathbf{h}_k \in D_k$ representa el estado de las unidades ocultas en la capa k .
- **Matrices de peso:** Ω_k describe la transformación lineal entre la capa k y la $k + 1$.
- **Vectores de sesgo (*bias*):** β_k representa el término de compensación de cada capa.
- **Función de activación:** $a[\cdot]$ es una función no lineal aplicada elemento a elemento.

El flujo de propagación de la información a través de la red se define mediante

¹Para evitar confusiones, cabe señalar que los subíndices i y o en las variables D_i y D_o provienen de los términos en inglés *input* y *ouput*, haciendo referencia a la dimensión de entrada y dimensión de salida, respectivamente.

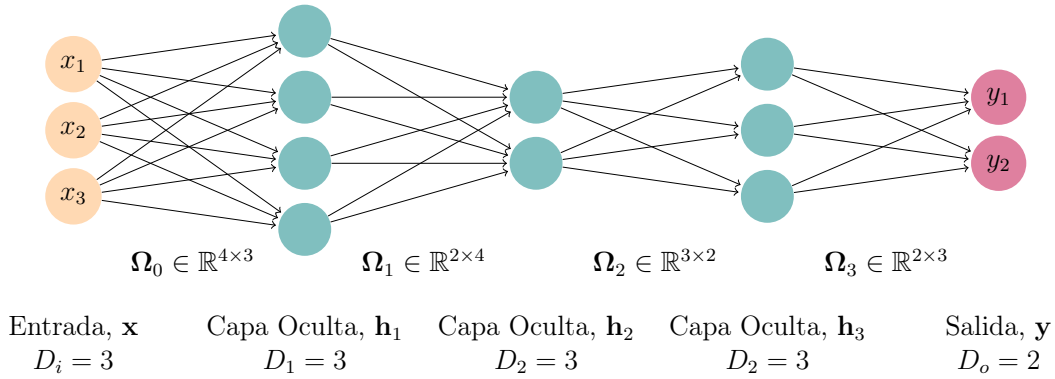


Figura 2.1: Notación para una red con una entrada $\mathbf{x}$ de dimensión $D_i = 3$, una salida $\mathbf{y}$ de dimensión $D_o = 2$ y $K = 3$ capas ocultas $\mathbf{h}_1$, $\mathbf{h}_2$ y $\mathbf{h}_3$ de dimensiones $D_1 = 4$, $D_2 = 2$ y $D_3 = 3$ respectivamente. Los pesos se almacenan en las matrices Ω_k que multiplican las activaciones de la capa precedente para generar las pre-activaciones de la siguiente capa. Por ejemplo, la matriz de pesos Ω_1 que computa las pre-activaciones en $\mathbf{h}_2$ a partir de las de $\mathbf{h}_1$ tiene dimensión 2×4 . Esta matriz se aplica a las 4 unidades ocultas de la primera capa y genera las entradas para las 2 unidades de la siguiente. Los sesgos se almacenan en los vectores β_k y tienen la dimensión de la capa que alimentan.

la siguiente secuencia de transformaciones:

$$\begin{aligned}
 \mathbf{h}_1 &= a[\beta_0 + \Omega_0 \mathbf{x}] \\
 \mathbf{h}_2 &= a[\beta_1 + \Omega_1 \mathbf{h}_1] \\
 &\vdots \\
 \mathbf{h}_K &= a[\beta_{K-1} + \Omega_{K-1} \mathbf{h}_{K-1}] \\
 \mathbf{y} &= \beta_K + \Omega_K \mathbf{h}_K
 \end{aligned}$$

En la capa k -ésima el vector de sesgo β_k tiene dimensión D_k para $k = 1, \dots, K - 1$, mientras que β_K tiene dimensión D_o . La primera matriz de pesos Ω_0 tiene dimensión $D_1 \times D_i$ y la última matriz de pesos tiene dimensión $D_o \times D_K$, mientras que en el resto de capas intermedias la matriz de pesos Ω_k tiene dimensión $D_{k+1} \times D_k$ (ver figura 2.1).

Cuando hablamos de aprendizaje o entrenamiento de un modelo, nos referimos al intento de encontrar los parámetros ϕ óptimos que permitan realizar predicciones de salida precisas y coherentes a partir de los datos de entrada. Para aprender estos parámetros, se utiliza un conjunto de datos de entrenamiento compuesto por pares de ejemplos de entrada y salida $\{(\mathbf{x}_n, \mathbf{y}_n)\}$. Nuestro objetivo es seleccionar aquellos parámetros que mapeen cada entrada de entrenamiento a su salida asociada de la forma más precisa posible. Para cuantificar el error introducimos la función de pérdida $L[\phi]$ ². En consecuencia, al entrenar el modelo buscamos el conjunto de

²No existe una única función de pérdida $L[\phi]$ sino que se define según el problema a resolver.

parámetros $\hat{\phi}$ que minimicen dicha función de pérdida, es decir:

$$\hat{\phi} = \arg \min_{\phi} [L[\phi]] \quad (2.1)$$

El operador $\arg \min$ nos indica que el resultado del problema de optimización no es el valor mínimo de la pérdida, sino el argumento ϕ con el cual se alcanza dicho valor mínimo.

2.2. Sotfmax

La función Sotfmax es una generalización de la función logística. Dado un vector $\mathbf{x}^T = (\mathbf{x}_1, \dots, \mathbf{x}_n)$, la función Sotfmax $\sigma : \mathbb{R}^n \rightarrow (0, 1)^n$ se define por componentes como:

$$\sigma[\mathbf{x}]_i = \frac{e^{\mathbf{x}_i}}{\sum_{j=1}^n e^{\mathbf{x}_j}} \quad \text{para } i = 1, \dots, n \quad (2.2)$$

y garantiza que $\sum_{i=1}^n \sigma[\mathbf{x}]_i = 1$.

Por lo general, y en el campo de la informática, la función Sotfmax también se aplica a matrices $\mathbf{X} \in \mathbb{R}^{N \times M}$. En este contexto, la función se aplica por filas, donde para cada elemento de la matriz se define como:

$$\sigma[\mathbf{X}]_{i,j} = \frac{e^{\mathbf{X}_{i,j}}}{\sum_{k=1}^M e^{\mathbf{X}_{i,k}}} \quad (2.3)$$

2.3. Normalización de Capa (*LayerNorm*)

Dado un vector $\mathbf{x}^T = (\mathbf{x}_1, \dots, \mathbf{x}_n) \in \mathbb{R}^n$, la función $\text{LayerNorm} : \mathbb{R}^n \rightarrow \mathbb{R}^n$ se define como:

$$\text{LayerNorm}[\mathbf{x}] = \gamma \odot \frac{\mathbf{x} - \mu \mathbf{1}}{\sqrt{\sigma^2 + \epsilon}} + \beta \quad (2.4)$$

donde $\mathbf{1} \in \mathbb{R}^n$ es el vector de unos, $\epsilon > 0$, $\odot$ denota el producto de Hadamard (multiplicación componente a componente), y $\gamma, \beta \in \mathbb{R}^n$ son vectores de parámetros aprendibles. Además:

$$\mu = \frac{1}{n} \sum_{j=1}^n \mathbf{x}_j \quad \text{y} \quad \sigma^2 = \frac{1}{n} \sum_{j=1}^n (\mathbf{x}_j - \mu)^2 \quad (2.5)$$

La normalización de capa es un operador fundamental diseñado para garantizar la estabilidad numérica de los datos a medida que atraviesan las múltiples capas de una red neuronal pues, en arquitecturas muy profundas, las multiplicaciones matriciales sucesivas pueden provocar que los valores de los vectores crezcan descontroladamente o tiendan a cero. La función LayerNorm mitiga este problema forzando que cada vector adquiera una media de cero y una varianza unitaria. Además, una de sus principales diferencias respecto a otras técnicas tradicionales basadas en lotes (*batches*), como *Batch Normalization*, es que se aplica de manera independiente sobre cada

token. Sin embargo, estandarizar estrictamente los datos podría limitar la capacidad expresiva de la red neuronal por lo que, para evitar esta pérdida de flexibilidad, se introducen parámetros que permiten reescalar y desplazar los valores si, durante el entrenamiento, el modelo descubre que una distribución diferente es más óptima para representar la información.

Transformador

La aparición de los transformadores (*transformer*) ha marcado un antes y un después en el campo del aprendizaje profundo (*deep learning*). Introducida a finales de 2017 por un equipo de investigadores de Google en el célebre artículo “*Attention is all you need*” [9], esta arquitectura desafió el paradigma dominante al prescindir por completo de las clásicas redes neuronales recurrentes (RNN) para el procesamiento de secuencias.

Su gran innovación radica en el mecanismo de autoatención (*self-attention*), el cual permite determinar qué partes de la información de entrada son más relevantes en cada momento. El nombre de transformador deriva precisamente de su función: transformar un conjunto de vectores iniciales en un nuevo conjunto de vectores de la misma dimensión. Sin embargo, esta nueva representación de la entrada es mucho más rica en contexto, significado e interpretación de los datos iniciales, lo que resulta fundamental para resolver con éxito tareas posteriores.

Las implicaciones que ha tenido esta nueva arquitectura han sido sorprendentes, pues aunque nacieron con el objetivo de revolucionar el Procesamiento de Lenguaje Natural (NLP), su impacto ha trascendido esta disciplina y forman la base de modelos de visión artificial y sistemas multimodales. Tal ha sido la magnitud de este hito tecnológico que los modelos extensos de lenguaje basan su diseño en esta arquitectura: la familia de modelos GPT (*Generative Pre-Trained Transformer*) de OpenAI, el modelo BERT (*Bidirectional Encoder Representations from Transformers*) y modelos como Gemini de Google entre otros.

Este capítulo pretende asentar y desgranar en detalle la arquitectura interna del transformador, analizando el funcionamiento de todos los ingredientes que lo componen y, sobre todo, el mecanismo de autoatención que hace posible el funcionamiento de este.

3.1. Procesamiento

La entrada de un transformador será un conjunto de vectores $\{\mathbf{x}_n\}$ de dimensión D_i donde $n = 1, \dots, N$. Dichos vectores recibirán el nombre de *tokens*, donde un *token* podrá representar una palabra dentro de una frase, partes de una palabra o

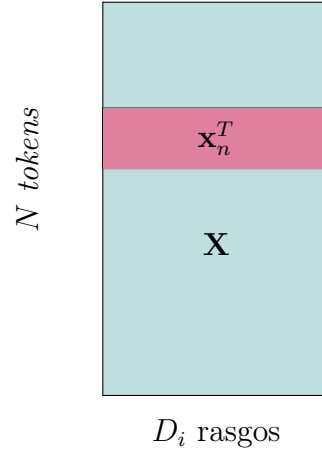


Figura 3.1: Esquema de la matriz de datos.

incluso de una imagen. Además, dado un *token* $\mathbf{x}_n^T = (\mathbf{x}_{n1}, \mathbf{x}_{n2}, \dots, \mathbf{x}_{nD_i})$ llamaremos a sus componentes $\mathbf{x}_{nm}$ rasgos.

Los vectores o *tokens* $\{\mathbf{x}_n\}$ los agruparemos en una matriz $\mathbf{X}$ de dimensión $N \times D_i$ donde la fila n -ésima representará el *token* $\mathbf{x}_n^T$ (ver 3.1).

El transformador será una función no lineal que mapea la matriz de entrada $\mathbf{X}$ hacia una nueva representación $\mathbf{Y}$ de la misma dimensión $N \times D_i$:

$$\mathbf{Y} = \text{Transformador}[\mathbf{X}] \quad (3.1)$$

La salida $\mathbf{Y}$ será capaz de capturar la relación que tienen los vectores de entrada gracias a un mecanismo de autoatención. Esto permitirá que cada vector resultante $\mathbf{y}_i^T$ de $\mathbf{Y}$ integre el contexto global de la secuencia, capturando la información relevante de todos los demás elementos.

Supongamos que nuestro conjunto inicial de *tokens* es $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N$ y queremos convertirlo en $\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_N$, de tal manera que cada uno de los $\mathbf{y}_n$ no solo dependa del vector de entrada $\mathbf{x}_n$ sino también de todos los demás vectores $\mathbf{x}_m$. La manera más elegante y sencilla de expresar esta dependencia es mediante una combinación lineal:

$$\mathbf{y}_n = \sum_{m=1}^N a_{nm} \mathbf{x}_m \quad n = 1, 2, \dots, N \quad (3.2)$$

donde los coeficientes a_{nm} recibirán el nombre de **coeficientes de atención**. Estos coeficientes a_{nm} tienen como objetivo representar la influencia de los vectores $\mathbf{x}_m$ sobre el vector de salida $\mathbf{y}_n$. Valores muy cercanos al cero indicarán poca influencia y valores más grandes indicarán una mayor influencia sobre aquel. Sin embargo, también queremos asegurar el hecho de que si una entrada particular “pone más atención” sobre el vector salida, entonces el resto de entradas deberán “poner menos atención” sobre esta. Matemáticamente:

$$a_{nm} \geq 0 \quad n, m = 1, 2, \dots, N \quad (3.3)$$

$$\sum_{m=1}^N a_{nm} = 1 \quad n = 1, 2, \dots, N \quad (3.4)$$

3.2. Coeficientes de atención

El objetivo es ahora determinar cómo calcular dichos **coeficientes de atención**. Iremos de menos a más, comenzando con un modelo sencillo al que, progresivamente, iremos añadiendo complejidad y detalles.

Para ponernos en situación, supongamos un grupo de filósofos sentados alrededor de una mesa redonda debatiendo y colaborando en la creación de una nueva teoría filosófica. Cada filósofo posee una etiqueta que identifica su corriente de pensamiento o especialidad, la cual actúa como **clave**, y un conocimiento profundo que puede aportar al grupo, que actúa como **valor**. Cuando un filósofo necesita inspiración o tiene dudas sobre algo formula una pregunta al resto de la mesa, la cual actúa como **consulta**. De todos los integrantes que hay en la mesa, el filósofo que lanza la pregunta necesita saber a quién debe “prestar más atención”, es decir, qué compañero tiene la especialidad que mejor encaje con su pregunta. En otras palabras, quiere encontrar a aquel filósofo cuya clave coincida mejor con su consulta. Sin embargo, como buen filósofo, no solo quiere tener en cuenta la respuesta de uno sino la de todos los filósofos en la mesa. Para ello, el filósofo comparará su consulta con todas las claves de sus compañeros e incluso con su propia clave (de donde deriva el término de autoatención) y asignará una distribución de pesos que medirán cuánta atención prestar a cada filósofo. Finalmente, cada filósofo le dará su respuesta, es decir, su valor, y el filósofo que lanza la pregunta considerará y ponderará estas respuestas según los pesos de atención. De esta manera, el filósofo tendrá en cuenta las respuestas de todos los participantes pero a algunos les hará más caso que a otros. Cabe destacar que este proceso es simultáneo, pues cada uno de los integrantes lanzará su propia pregunta al resto de filósofos, actualizando y enriqueciendo su conocimiento al mismo tiempo.

En una primera aproximación simplificada, supondremos N filósofos en la mesa. Cada uno de ellos será representado mediante un vector $\mathbf{x}_m$. Las consultas y las claves serán representados mediante los vectores $\mathbf{q}_m$ y $\mathbf{k}_m$ respectivamente, mientras que el valor será el propio filósofo, es decir, el vector $\mathbf{x}_m$. Dado un filósofo $\mathbf{x}_n$, será necesario comparar su correspondiente consulta $\mathbf{q}_n$ con las claves $\mathbf{k}_m$ de los filósofos presentes en la mesa. Para realizar la comparación y medir el grado de similitud emplearemos el producto escalar, por lo que podríamos definir, inicialmente, los coeficientes de atención como:

$$a_{nm} = \mathbf{q}_n^T \cdot \mathbf{k}_m \quad (\text{consulta} \cdot \text{clave}) \quad (3.5)$$

donde $\cdot$ denota el producto escalar. Sin embargo, al no cumplir (3.3) ni (3.4), es necesario aplicar la función Softmax sobre el conjunto de productos escalares obtenidos. Se definen entonces los coeficientes de atención como:

$$a_{nm} := \text{Softmax}_m(\mathbf{q}_n^T \mathbf{k}_m) = \frac{\exp(\mathbf{q}_n^T \mathbf{k}_m)}{\sum_{m'=1}^N \exp(\mathbf{q}_n^T \mathbf{k}_{m'})} \quad m = 1, \dots, N \quad (3.6)$$

donde cada uno de estos coeficientes representaría la atención que pone el filósofo $\mathbf{x}_n$, el que realiza la consulta, sobre el filósofo $\mathbf{x}_m$, el que puede responder a su consulta.

Según comentábamos, el proceso es simultáneo y cada vector $\mathbf{x}_n$ de $\mathbf{X}$, mediante (3.2), genera su propio vector $\mathbf{y}_n$, que, conceptualmente y aplicado a la analogía, representa la conclusión a la que llega el filósofo $\mathbf{x}_n$.

Todas estas ecuaciones, (3.2) y (3.6), se pueden expresar matricialmente como:

$$\mathbf{Y} = \text{Softmax}[\mathbf{QK}^T]\mathbf{X} \quad (3.7)$$

donde $\mathbf{Y}$ tiene la misma dimensión $N \times D_i$ que $\mathbf{X}$ y cuyas filas son los vectores de salida $\mathbf{y}_1^T, \mathbf{y}_2^T, \dots, \mathbf{y}_m^T$.

Si observamos la ecuación (3.7), se aprecia que la transformación de $\{\mathbf{x}_m\}$ a $\{\mathbf{y}_m\}$ es rígida, ya que hemos asumido que las consultas y las claves son valores fijos que no dependen de la entrada $\mathbf{X}$. Bajo esta premisa, cada filósofo aporta siempre el mismo valor independientemente del filósofo, lo que otorga al modelo muy poca flexibilidad para decidir, en función de la situación, a quién debe prestar más atención. Es por ello que nos gustaría que el modelo tuviese capacidad de aprendizaje, es decir, que no se limite a una transformación fija, sino que aprenda a ajustar las representaciones de las consultas, las claves y los valores durante su entrenamiento. Para ello introduciremos hiperparámetros en el cálculo de las consultas, las claves y los valores.

3.3. Consulta, Clave y Valor

Como comentábamos al final de la sección anterior, la ecuación (3.7) carece de parámetros y no hay capacidad de aprendizaje. Nos interesará entonces transformar la entrada $\mathbf{X}$ mediante alguna transformación lineal:

$$\tilde{\mathbf{X}} = \mathbf{X}\mathbf{\Omega} + \boldsymbol{\beta} \quad (3.8)$$

donde $\mathbf{\Omega}$ es una matriz de pesos de dimensión $D_i \times D_i$ y $\boldsymbol{\beta}$ son los sesgos, como si de una red neuronal de una sola capa se tratase¹. De esta manera, nos interesaría que el modelo fuese capaz de aprender características independientes para la consulta; distintas características para las claves y, también, otras características para los valores. En consecuencia, definiremos 3 matrices de pesos independientes con su correspondiente transformación lineal²:

$$\mathbf{Q} = \mathbf{X}\mathbf{\Omega}_q \quad (3.9)$$

$$\mathbf{K} = \mathbf{X}\mathbf{\Omega}_k \quad (3.10)$$

$$\mathbf{V} = \mathbf{X}\mathbf{\Omega}_v \quad (3.11)$$

¹La ecuación (3.8) se puede expresar matricialmente como $\tilde{\mathbf{X}} = [\mathbf{X} \ 1] \begin{bmatrix} \mathbf{\Omega} \\ \boldsymbol{\beta} \end{bmatrix}$. De aquí en adelante y con el fin de aligerar la notación asumiremos que los sesgos están implícitos en la propia matriz de pesos $\mathbf{\Omega}$ y (3.8) equivadrá a $\tilde{\mathbf{X}} = \mathbf{X}\mathbf{\Omega}$.

²Se emplean las letras $\mathbf{Q}$, $\mathbf{K}$ y $\mathbf{V}$ por su terminología en inglés: *Query* (consulta), *Key* (clave) y *Value* (valor).

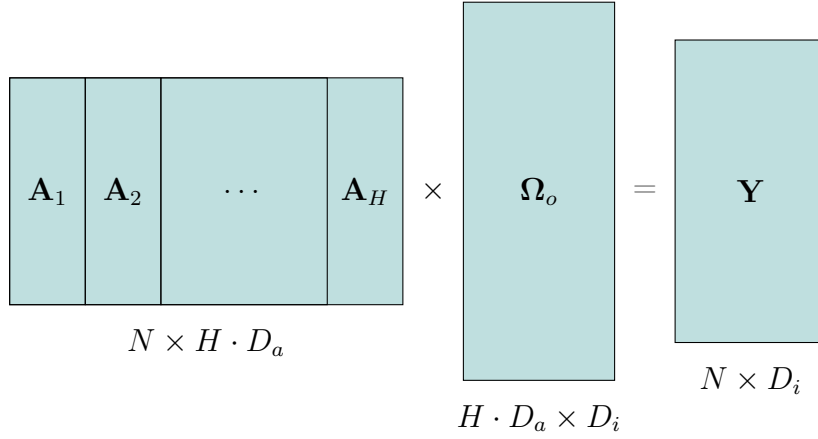


Figura 3.2: Concatenacion de capas de atención

Los parámetros de las matrices de pesos Ω_q , Ω_k y Ω_v irán ajustándose durante el entrenamiento de manera independiente, consiguiendo así esa flexibilidad que buscábamos. De esta manera, juntando (3.9), (3.10) y (3.11) con (3.7) obtenemos :

$$\mathbf{Y} = \text{Softmax}[\mathbf{Q}\mathbf{K}^T]\mathbf{V} \quad (3.12)$$

Las matrices Ω_q , Ω_k y Ω_v tendrán dimension $D_i \times D_i$ con el fin de garantizar la correcta multiplicación de las matrices $\mathbf{Q}$ y $\mathbf{K}$, y asegurar que la salida $\mathbf{Y}$ tenga la misma dimension $N \times D_i$ que $\mathbf{X}$.

Este mecanismo recibe el nombre, en inglés, de *dot-product self-attention* o de *attention-head*. Nosotros lo llamaremos *capa de autoatención* o *mecanismo de autoatención* y lo denotaremos por:

$$\mathbf{Y}[\mathbf{X}] = \text{Atención}[\mathbf{Q}, \mathbf{K}, \mathbf{V}] \quad (3.13)$$

En una red neuronal convencional cada entrada se multiplica por un peso fijo, de modo que si la red decide ignorar una entrada, lo hace para todos los datos. Por el contrario, con este nuevo mecanismo, cada activación se multiplica por el coeficiente de atención **dependiente** de los datos de entrada. Esta distinción permite al modelo prestar más atención a unas entradas que a otras según el contexto (entradas).

3.4. Atención Múltiple

En ocasiones, suele haber múltiples mecanismos o patrones distintos a los que hay que prestar atención en un mismo problema. Por ejemplo, en el lenguaje natural algunos patrones suelen estar asociados al tiempo verbal, como el sufijo “-ed” del inglés que hace referencia al pasado; mientras que otros patrones tienen relación con la semántica o el significado de las palabras. Por ello, resulta interesante tener

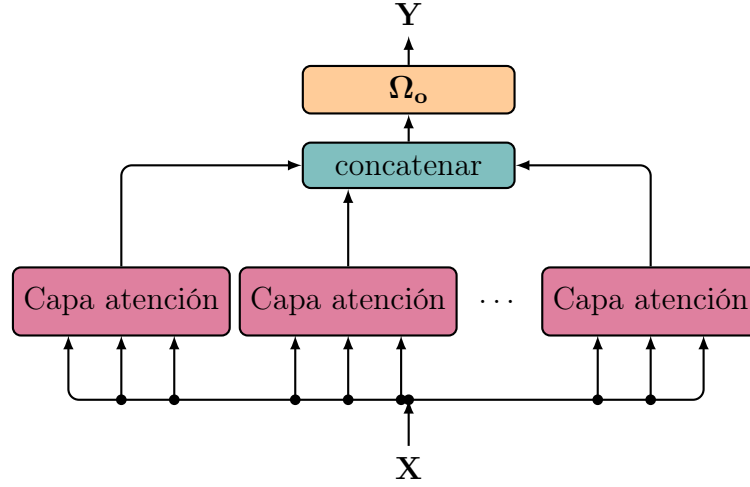


Figura 3.3: Flujo de datos en la atención múltiple.

varias capas de autoatención trabajando simultáneamente y cada una de ellas con sus propias matrices de consulta, clave y valor.

Supongamos que tenemos H capas de atención independientes de dimensión $N \times D_i$:

$$\mathbf{A}_h[\mathbf{X}] = \text{Atención}[\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h] \quad (3.14)$$

donde las matrices de consulta, clave y valor son independientes, es decir:

$$\mathbf{Q}_h = \mathbf{X}\Omega_q^h \quad (3.15)$$

$$\mathbf{K}_h = \mathbf{X}\Omega_k^h \quad (3.16)$$

$$\mathbf{V}_h = \mathbf{X}\Omega_v^h \quad (3.17)$$

para cada $h = 1, 2, \dots, H$. Con todo ello, concatenaremos las capas $\mathbf{A}_h$ y aplicaremos una transformación lineal mediante una matriz Ω_o cuyos elementos se irán ajustando también durante el entrenamiento:

$$\mathbf{Y} = \text{Concatenar}[\mathbf{A}_1, \mathbf{A}_2, \dots, \mathbf{A}_H]\Omega_o \quad (3.18)$$

donde la matriz resultante de concatenar las matrices $\mathbf{A}_h$ tiene dimensión $N \times H \cdot D_i$ y, en consecuencia, Ω_o tiene dimensión $H \cdot D_i \times D_i$ conservando así la dimensión $N \times D_i$ de $\mathbf{X}$.

En la práctica, cada una de estas capas de autoatención $\mathbf{A}_h$, tiene una dimensión inicial $N \times D_a$ donde $D_a = \frac{D_i}{H}$ con el objetivo de que la matriz concatenada tenga dimensión $N \times D_i$ (ver figura 3.2). Además, de esta manera el coste computacional total de evaluar las H capas en paralelo se mantiene aproximadamente igual al coste que tendría evaluar una única capa de atención. Esto evita que el coste de las operaciones matriciales se dispare al utilizar múltiples capas.

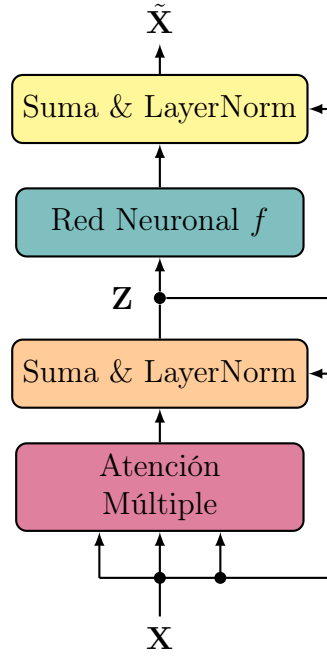


Figura 3.4: Arquitectura del transformador.

3.5. Transformador: Arquitectura

La atención múltiple definida en este último apartado es el eje central de la arquitectura en un transformador. Sin embargo, es bien sabido que las redes neuronales suelen mejorar su desempeño a medida que aumentan en profundidad, es decir, en número de capas, y los transformadores no son una excepción.

Para mejorar el entrenamiento y evitar que el gradiente se desvanezca en la profundidad de la red, implementamos una *conexión residual*: una conexión que permite que la información de una capa anterior se salte las operaciones intermedias para sumarse directamente a la salida de estas. Esta suma se puede hacer gracias a que las *capas de autoatención* no modifican la dimensión de la entrada $\mathbf{X}$. Después se aplica una normalización de capas que ayuda a estabilizar y acelerar el aprendizaje:

$$\mathbf{Z} = \text{LayerNorm}[\mathbf{Y}[\mathbf{X}] + \mathbf{X}] \quad (3.19)$$

Posteriormente, y con el objetivo de aumentar la no-linealidad del modelo, pasamos $\mathbf{Z}$ por una red neuronal $\mathbf{y} = f[\mathbf{z}, \phi]$ donde $\mathbf{x}, \mathbf{y} \in \mathbb{R}^{D_i}$:

$$\tilde{\mathbf{X}} = \text{LayerNorm}[f[\mathbf{Z}, \phi] + \mathbf{Z}] \quad (3.20)$$

Y esta sería la forma final de un transformador.

Habitualmente, la arquitectura de estos modelos no se compone de un único bloque, sino que se construye concatenando múltiples transformadores apilados, donde la salida de uno constituye la entrada del siguiente. Desde su introducción original [9], el modelo fue diseñado expresamente apilando 6 transformadores idénticos, pero aunque estas capas comparten exactamente la misma estructura, cada capa utiliza

parámetros, pesos y sesgos completamente independientes que el modelo optimiza de manera individual durante el entrenamiento.

El hecho de que cada capa aprenda parámetros distintos permite que cada nivel se “especialice” en un grado de abstracción diferente conforme la información avanza por el modelo. Las primeras capas tienden a enfocarse en relaciones locales y patrones sintácticos básicos, mientras que las capas más profundas utilizan esa base para construir representaciones semánticas globales y complejas. Si todas las capas compartieran los mismos parámetros el modelo se limitaría a aplicar siempre la misma transformación, lo que limitaría su capacidad de aprendizaje.

El éxito de esta arquitectura se determinó y verificó de manera empírica. Ya en el artículo original los autores demostraron mediante diversas pruebas que los modelos más grandes y profundos ofrecían una mejor calidad en las tareas de traducción. En la actualidad, esta observación ha evolucionado hasta consolidarse en la denominada hipótesis del escalado (*scaling hypothesis*) [3]. Esta hipótesis postula que, al aumentar a gran escala el número de parámetros y capas del modelo, junto a bases de datos enormes, se logran incrementos en el rendimiento sin necesidad de alterar la arquitectura básica del transformador. Como evidencia empírica contemporánea de este fenómeno, modelos de lenguaje de gran tamaño como GPT-3 han llevado este concepto al extremo, utilizando una pila de 96 transformadores.

Capítulo 4

Lenguaje Natural

Tras explorar en el capítulo anterior la arquitectura del transformador y su mecanismo de autoatención, nos adentramos ahora en su mayor caso de éxito: la revolución del Procesamiento del Lenguaje Natural (NLP). El diseño del transformador superó las limitaciones de los modelos clásicos, permitiendo procesar grandes secuencias de texto en paralelo y capturar el contexto de las palabras con una eficacia sin precedentes.

Sin embargo, el verdadero punto de inflexión llegó de la mano de OpenAI en su influyente artículo *Scaling Laws for Neural Language Models* [3], donde demostraron empíricamente que el rendimiento de los modelos de lenguaje mejoraban según aumentaban el tamaño de la red, la capacidad de cómputo y, sobre todo, el volumen de los datos. Al llevar este escalado a niveles masivos, nacieron los *Large Language Models* (LLMs) o, en español, los modelos extensos de lenguaje. Gracias a este crecimiento, se observó que redes entrenadas únicamente para predecir la siguiente palabra desarrollaron una profunda “comprensión” de la sintaxis y la semántica y dejaron de ser sistemas especializados en una tarea para convertirse en herramientas de propósito general, capaces de traducir, resumir o razonar con apenas unas instrucciones (los famosos *prompts*).

Con todo ello, resulta fundamental entender qué ocurre exactamente en el interior de estos modelos extensos del lenguaje y cómo se utilizan según el problema que queramos resolver. En este capítulo explicaremos en detalle cómo se procesa el lenguaje natural dentro de los LLMs; abordaremos cómo se transforma el texto bruto en vectores mediante la tokenización y los *embeddings*, y analizaremos varias estructura de los transformadores, entre ellas, las estructuras autorregresivas que impulsan la actual inteligencia artificial generativa.

4.1. Tokenización

El primer paso en el procesamiento del lenguaje es transformar el texto en bruto en un formato estructurado que el modelo pueda interpretar. Para ello, es necesario definir un **vocabulario**: un conjunto finito de unidades básicas de texto que el modelo es capaz de reconocer y procesar.

Una primera aproximación para definir este vocabulario consistiría en utilizar únicamente caracteres individuales: letras, números y signos de puntuación. Sin embargo, este enfoque presenta desventajas significativas pues aislar los caracteres destruye la semántica del lenguaje y dificulta la comprensión real del texto, por no mencionar que genera secuencias de entrada excesivamente largas (por ejemplo, este pequeño párrafo contiene ya 439 caracteres). Otra aproximación sería considerar un diccionario literal compuesto exclusivamente por palabras enteras. No obstante, este método es poco flexible, pues si el modelo recibiese una palabra que no está en su vocabulario, como un nombre propio o un error ortográfico, el modelo sería incapaz de procesarlo.

Para resolver esta dicotomía, se emplea el proceso de **tokenización**. Este método busca un equilibrio dividiendo el texto de entrada en unidades más pequeñas llamadas *tokens*, que formarán el vocabulario y serán lo que realmente reciba y procese el transformador como entrada. Estos *tokens* pueden ser palabras enteras que suelen aparecer con mucha frecuencia, caracteres individuales o fragmentos de palabras (raíces o terminaciones). Para entender las ventajas de usar *tokens* consideremos las palabras *cocinar*, *cocinado*, *cocinero* y *cocina*. Todas ellas comparten la misma raíz *cocin-*. Mediante la tokenización, resulta mucho más práctico considerar *cocin-* como un *token* independiente perteneciente al vocabulario y, de este modo, interpretar el resto de palabras como la concatenación de dicha raíz con sus correspondientes sufijos que, a su vez, serán *tokens* independientes o concatenaciones de otros *tokens*.

Este proceso de tokenización otorga al modelo la flexibilidad deseada pues en lugar de fallar ante palabras nuevas o desconocidas, el modelo los divide en *tokens* que sí pertenecen a su vocabulario y que, en consecuencia, sí es capaz de procesar.

Surge entonces la cuestión fundamental de cómo se definen y obtienen dichos *tokens*. Existen varios métodos, pero el más famoso y el más utilizado es mediante el algoritmo de **Codificación de Pares de Bytes** (*Byte Pair Encoding*):

Algoritmo 1 Algoritmo de codificación de pares de bytes (BPE)

Entrada: Cadena de texto $\mathcal{S}$ y tamaño de vocabulario requerido k .

Salida: Vocabulario $\mathcal{V}$ de *tokens*.

- 1: Inicializar $\mathcal{V}$ como el conjunto de todos los caracteres individuales únicos en $\mathcal{S}$.
 - 2: Reemplazar todos los espacios en $\mathcal{S}$ con un *token* especial de espacio.
 - 3: **while** $|\mathcal{V}| < k$ **do**
 - 4: Encontrar el par de *tokens* adyacentes t_1, t_2 más frecuente en $\mathcal{S}$.
 - 5: Definir un nuevo *token* $t_{\text{nuevo}} = \text{Concatenar}(t_1, t_2)$.
 - 6: Añadir t_{nuevo} a $\mathcal{V}$.
 - 7: Reemplazar todas las ocurrencias del par t_1, t_2 en $\mathcal{S}$ con t_{nuevo} .
 - 8: **end while**
 - 9: **return** $\mathcal{V}$
-

La idea del algoritmo es la siguiente: partimos de un texto $\mathcal{S}$ y un valor k que representará el tamaño de nuestro vocabulario $\mathcal{V}$, donde los *tokens* iniciales que componen $\mathcal{V}$ son todos los caracteres distintos que aparecen en $\mathcal{S}$. En cada iteración, el algoritmo busca el par de *tokens* consecutivos que aparece con mayor frecuencia

Hoy ya es ayer y ayer ya es hoy, ya llegó el día, y hoy es hoy
Hoy ya es ayer y ayer ya es hoy, ya llegó el día, y hoy es hoy
Hoy ya es ayer y ayer ya es hoy, ya llegó el día, y hoy es hoy
Hoy ya es ayer y ayer ya es hoy, ya llegó el día, y hoy es hoy
Hoy ya es ayer y ayer ya es hoy, ya llegó el día, y hoy es hoy

Figura 4.1: Ejemplo de 5 iteraciones de BPE sobre una frase. Inicialmente el vocabulario $\mathcal{V}$ está formado por los caracteres que componen la frase. En la primera iteración se busca el par de *tokens* consecutivos más frecuente que, en este caso, son “o” e “y”. Estos dos nuevos *tokens* forman un *token* nuevo “oy” que se añade a $\mathcal{V}$. Después, en la segunda iteración, se tiene que los *tokens* “e” y “s” son el par consecutivo más frecuente, y, en consecuencia, se añade un nuevo *token* “es” al vocabulario. En la tercera iteración, el par de *tokens* consecutivos más frecuente son “h” y “oy”, luego, repitiendo el mismo proceso, se añade un nuevo *token* “hoy” a $\mathcal{V}$. Finalmente, el vocabulario resultante será la unión de los caracteres que conforman la frase inicial con el conjunto {oy, es, hoy, ya}. Destacar que no hay un criterio estandarizado para resolver los empates en las frecuencias y en este ejemplo, en caso de empate, se ha elegido aleatoriamente el par más frecuente.

y acto seguido los fusiona para crear un *token* nuevo que se añade al vocabulario y actualiza el texto original reemplazando el par separado con el nuevo *token*. Este proceso se repite sucesivamente durante k iteraciones (ver figura 4.1).

Observamos que este algoritmo depende mucho del texto de entrada $\mathcal{S}$, por lo que es imprescindible partir de un corpus de entrenamiento masivo que represente el dominio sobre el que operará el modelo. Por ejemplo, si el objetivo es desarrollar un sistema especializado en derecho, resulta útil que el corpus esté compuesto por la mayor cantidad de textos jurídicos. De este modo, el algoritmo aprenderá a agrupar los caracteres en raíces y terminaciones más frecuentes de dicha jerga.

En la práctica el vocabulario $\mathcal{V}$ es un diccionario clave-valor donde cada *token* tiene asociado un identificador numérico único. Además, es muy importante tener en cuenta que el carácter que representa el espacio en blanco no se omite, sino que se incluye en $\mathcal{V}$ como un *token* inicial. De esta manera el modelo puede aprender a distinguir si una secuencia de caracteres forma una palabra independiente o si es un fragmento. Generalmente, el espacio en blanco suele estar concatenado al inicio de ciertos *tokens* (ver figura 4.2).

4.2. *Embeddings*

Una vez definido el vocabulario y comprendido el proceso de tokenización, el siguiente paso natural es convertir dichos *tokens* en un formato que la red neuronal pueda procesar. Como se ha comentado en la sección anterior, los *tokens* pertenecientes a un vocabulario $\mathcal{V}$ son, inicialmente, simples identificadores numéricos. Sin embargo, tal y como se introdujo al inicio de la sección 3.1, el transformador opera con vectores $\mathbf{x}_n^T = (\mathbf{x}_{n1}, \mathbf{x}_{n2}, \dots, \mathbf{x}_{nD_i})$ a los que también llamábamos *tokens*. Con todo ello, es necesario realizar una transformación que asigne a un identificador un

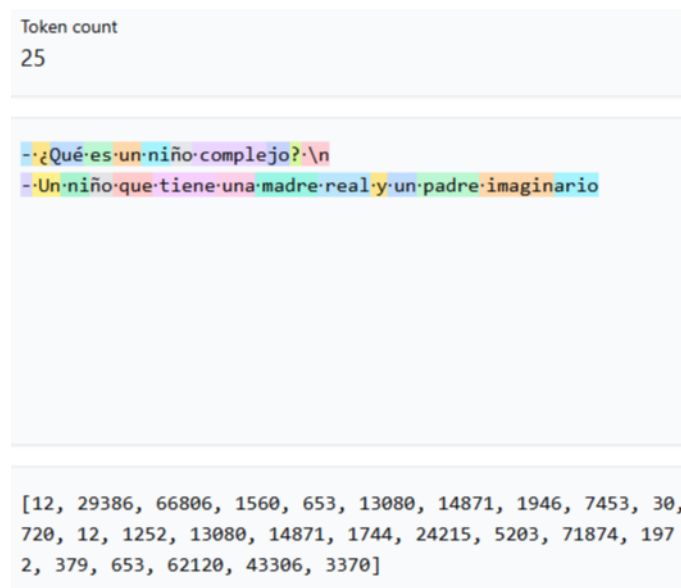
(a) Vocabulario GPT2 de 50,000 *tokens*(b) Vocabulario *cl100k_base* de 100,000 *tokens*

Figura 4.2: Comparación visual de la fragmentación de tokens entre los vocabularios de GPT-2 y GPT-4 (*cl100k_base*). El número de *tokens* para la misma frase es distinto. También se observa que en a) la palabra *madre* se compone de dos *tokens*, mientras que en b) es un *token* entero. En ambos el punto representa el espacio en blanco y los *tokens* que suelen representar el inicio de una palabra van precedidos por el punto. Los vectores de números representan los identificadores de cada *token* en orden y se observa que *tokens* iguales como “·¿” tienen distinto identificador: 1587 en a) y 29386 en b). Imágenes obtenidas a través de <https://tiktokenizer.chatgptcn.com/>, herramienta interactiva donde se puede observar cómo tokenizan distintos vocabularios.

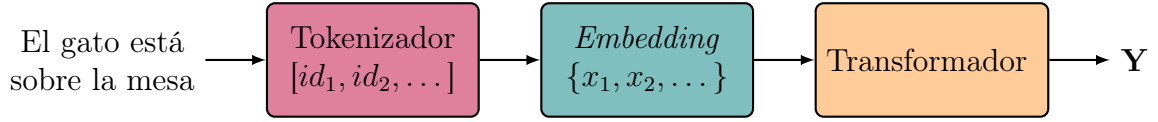


Figura 4.3: Procesamiento del texto hasta llegar al transformador.

vector.

Supongamos entonces un vocabulario $\mathcal{V}$ de cardinal k . Definiremos una matriz $\mathbf{E}$, denominada matriz de *embedding*, de dimensión $D_i \times k$. Además, cada *token* de $\mathcal{V}$ será representado mediante un vector $t_n^T = (0, \dots, 1, \dots, 0) \in \mathbb{R}^k$ formado por 0's salvo la posición n -ésima donde vale 1 y representará el *token* n -ésimo. Finalmente, realizamos la siguiente transformación lineal:

$$\mathbf{x}_n = \mathbf{E}t_n \in \mathbb{R}^{D_i} \quad (4.1)$$

donde $\mathbf{x}_n$ es el *token* n -ésimo que procesará el transformador y al que hacíamos referencia en la sección 3.1.

Cabe señalar que representar los *tokens* de $\mathcal{V}$ solo con vectores de la base canónica no es práctico, pues, por lo general, $\mathcal{V}$ suele estar compuesto por cientos de miles de *tokens* y los vectores t_n serían dimensionalmente muy grandes. Además, lo habitual es considerar $D_i \leq k$, reduciendo así la dimensión de $\mathbf{x}_n$. Por otro lado, los vectores canónicos no capturan ningún tipo significado o semántica y mucho menos relaciones entre *tokens*, ya que se trata de vectores ortogonales entre sí. En consecuencia, la matriz $\mathbf{E}$ se define como una matriz de parámetros entrenables y cuyos pesos se van ajustando durante el entrenamiento junto al transformador. De aquí en adelante cuando hagamos referencia a los *tokens* pensaremos siempre en (4.1).

4.3. Transformadores

Hasta ahora hemos visto cómo la información se procesa y convierte en vectores de $\mathbb{R}^{D_i}$ para que el transformador sea capaz de asimilarla e interpretarla y, sobre todo, comprendemos la arquitectura general del transformador. En consecuencia, en esta sección abordaremos cómo se adapta la arquitectura y tratamos esa información preprocesada dependiendo de la tarea específica que queramos resolver. Para ello, estudiaremos tres variantes principales: codificador (*encoder*), decodificador (*decoder*) y codificador-decodificador (*encoder-decoder*).

El codificador (*encoder*) procesa la secuencia de entrada en su totalidad para extraer una representación interna de alta dimensionalidad y actúa como un extractor de características diseñado para clasificar o analizar texto, tales como el análisis de sentimientos o el reconocimiento de entidades.

Por otro lado, el decodificador (*decoder*) es un modelo puramente generativo cuya finalidad es la construcción de nuevas secuencias de forma autorregresiva. Para garantizar un aprendizaje coherente, esta variante emplea un mecanismo de atención enmascarada que impide al modelo acceder a información de posiciones futuras. De este modo, la predicción de cada nuevo elemento se realiza basándose exclusivamente en los tokens generados con anterioridad.

Finalmente, la arquitectura codificador-decodificador (*encoder-decoder*) es una estructura híbrida de las anteriores concebida específicamente para problemas de secuencia a secuencia. En este esquema, el proceso se divide en dos fases complementarias: mientras el codificador genera una representación matemática global de la entrada, el decodificador produce la salida apoyándose no solo en su propia entrada, sino también en la información estructurada que le transfiere el codificador a través del mecanismo de atención cruzada (*cross-attention*). Esta interconexión es la que permite resolver con éxito tareas de alta complejidad donde el significado debe preservarse íntegramente entre distintos dominios, siendo la traducción automática de idiomas el ejemplo más representativo de su aplicación.

4.3.1. Decodificador

Los decodificadores son un tipo de arquitectura cuyo propósito principal es funcionar como modelos generativos para crear o generar secuencias de *tokens*. Pertenecen a la familia de los denominados *modelos autorregresivos*: modelos probabilísticos diseñados para procesar datos secuenciales, donde la probabilidad de que aparezca un elemento en la secuencia depende de los elementos que lo precedieron.

4.3.1.1. Probabilidad Condicionada

El objetivo principal de un modelo autoregresivo es, dada una secuencia de *tokens* $\mathbf{x}_1, \dots, \mathbf{x}_n$, calcular la probabilidad conjunta $p(\mathbf{x}_1, \dots, \mathbf{x}_n)$. Esta probabilidad, desarrollando mediante la probabilidad condicionada, resulta en:

$$p(\mathbf{x}_1, \dots, \mathbf{x}_n) = \prod_{t=1}^n p(\mathbf{x}_t | \mathbf{x}_1, \dots, \mathbf{x}_{t-1}) \quad (4.2)$$

Por lo tanto, calcular la probabilidad conjunta de toda la secuencia se traduce en calcular las correspondientes probabilidades condicionadas. Y esta es exactamente la tarea fundamental que realizan los modelos autorregresivos y, en nuestro caso, el decodificador.

Nuestro objetivo es, por tanto, adaptar la salida del transformador para que funcione como un modelo autorregresivo. En otras palabras, buscamos traducir su salida a una distribución de probabilidades sobre un vocabulario $\mathcal{V}$. Para ello, recordemos que a nivel interno el transformador mapea una secuencia de *tokens* $\mathbf{x}_1, \dots, \mathbf{x}_n$ en una secuencia de vectores $\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_n$ de dimensión D_i . Sin embargo, para que el decodificador pueda predecir el siguiente elemento, necesitamos que su salida sea una distribución sobre un vocabulario $\mathcal{V}$ de tamaño k , la cual determine la probabilidad que tiene cada *token* de continuar la secuencia, es decir, necesitamos modificar la salida del transformador en una distribución de probabilidad. Para lograr esto, realizaremos una transformación lineal utilizando una matriz de pesos $\mathbf{\Omega}_o$ de dimensión $D_i \times k$, seguida de la función de activación Softmax:

$$\mathbf{Y} = \text{Softmax}[\tilde{\mathbf{X}}\mathbf{\Omega}_o] \quad (4.3)$$

donde $\mathbf{Y}$ tiene dimensión $n \times k$ y $\tilde{\mathbf{X}}$ es la matriz cuyas filas son la salida $\tilde{\mathbf{x}}_1^T, \dots, \tilde{\mathbf{x}}_n^T$ del transformador. Al aplicar la función Softmax, garantizamos que cada una de

las filas $\mathbf{y}_j^T$ contenga elementos comprendidos entre 0 y 1, y que su suma total sea exactamente uno. Dicho de otra manera, cada fila se transforma en una distribución de probabilidad válida. Por lo tanto, la fila j -ésima $\mathbf{y}_j^T$ de $\mathbf{Y}$ representa la distribución de probabilidades sobre el vocabulario $\mathcal{V}$ para predecir el siguiente *token*, condicionada por la subsecuencia de *tokens* procesada hasta ese momento $\mathbf{x}_1, \dots, \mathbf{x}_j$ (con $j \leq n$):

$$\mathbf{Y} = \begin{pmatrix} p(\mathbf{x}_2 = v_1 | \mathbf{x}_1) & p(\mathbf{x}_2 = v_2 | \mathbf{x}_1) & \dots & p(\mathbf{x}_2 = v_k | \mathbf{x}_1) \\ p(\mathbf{x}_3 = v_1 | \mathbf{x}_1, \mathbf{x}_2) & p(\mathbf{x}_3 = v_2 | \mathbf{x}_1, \mathbf{x}_2) & \dots & p(\mathbf{x}_3 = v_k | \mathbf{x}_1, \mathbf{x}_2) \\ \vdots & \vdots & \ddots & \vdots \\ p(\mathbf{x}_{n+1} = v_1 | \mathbf{x}_1, \dots, \mathbf{x}_n) & p(\mathbf{x}_{n+1} = v_2 | \mathbf{x}_1, \dots, \mathbf{x}_n) & \dots & p(\mathbf{x}_{n+1} = v_k | \mathbf{x}_1, \dots, \mathbf{x}_n) \end{pmatrix}$$

donde $\mathbf{x}_{n+1}$ representa la predicción del siguiente *token* a toda la secuencia $(\mathbf{x}_1, \dots, \mathbf{x}_n)$.

Sin embargo, si observamos detenidamente la matriz $\mathbf{Y}$ y recordamos cómo funciona el proceso de autoatención, nos damos cuenta de un problema estructural: no podemos asegurar que las probabilidades condicionadas dependan exclusivamente de los *tokens* procesados hasta la posición j . Dado que la atención procesa toda la secuencia en paralelo y atiende a todos los elementos simultáneamente, al calcular la salida $\tilde{\mathbf{x}}_j$, el modelo tendría acceso a la información de los *tokens* futuros $(\mathbf{x}_{j+1}, \dots, \mathbf{x}_n)$. Si entrenásemos al modelo así, simplemente aprendería a hacer trampas: “miraría” el texto adelantado y copiaría la respuesta en lugar de aprender a predecirla. Para evitar esto y garantizar que la arquitectura funcione correctamente como un modelo autorregresivo, es necesario introducir una modificación en el proceso: la autoatención enmascarada (*masked self-attention*).

4.3.1.2. Autoatención Enmascarada

Para garantizar matemáticamente que cada fila j de la matriz $\mathbf{Y}$ represente una verdadera distribución condicionada de manera exclusiva a los *tokens* previos, debemos impedir que el *token* $\mathbf{x}_j$ “mire hacia el futuro”. Queremos asegurar que, al procesar la secuencia, un *token* en la posición j solo calcule su atención basándose única y exclusivamente en sí mismo y en los *tokens* que lo preceden, ignorando por completo la posición $j + 1$ y posteriores. Dado que los *tokens* fluyen de manera independiente por el transformador y el único punto donde intercambian información es durante el cálculo de los coeficientes de atención $(\mathbf{Q}\mathbf{K}^T)$, es precisamente en esta operación donde debemos intervenir.

Se introduce entonces una matriz de enmascaramiento, denotada como $\mathbf{M}$, de dimensión $N \times N$, donde N representaba el número de *tokens* que recibía el transformador como entrada. Esta matriz presenta la siguiente estructura:

$$\mathbf{M} = \begin{pmatrix} 0 & -\infty & -\infty & \dots & -\infty \\ 0 & 0 & -\infty & \dots & -\infty \\ 0 & 0 & 0 & \dots & -\infty \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 0 \end{pmatrix}$$

La matriz $\mathbf{M}$ se suma a la atención calculada justo antes de aplicar la función Softmax:

$$\text{Softmax} [\mathbf{Q}\mathbf{K}^T + \mathbf{M}]$$

Al realizar esta operación, observamos que las posiciones correspondientes a *tokens* futuros (la triangular superior) adquieren un valor $-\infty$ y, tras la función Softmax, dichas posiciones se anulan por completo. De esta manera la matriz con los coeficientes de atención a_{nm} resulta en:

$$\begin{array}{c} \mathbf{x}_1 \quad \mathbf{x}_2 \quad \mathbf{x}_3 \quad \cdots \quad \mathbf{x}_N \\ \mathbf{x}_1 \quad \left(\begin{array}{ccccc} a_{11} & 0 & 0 & \cdots & 0 \\ a_{21} & a_{22} & 0 & \cdots & 0 \\ a_{31} & a_{32} & a_{33} & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{N1} & a_{N2} & a_{N3} & \cdots & a_{NN} \end{array} \right) \end{array}$$

El *token* $\mathbf{x}_1$ solo presta atención a sí mismo, mientras que el segundo *token* $\mathbf{x}_2$ solo presta atención a sí mismo y a $\mathbf{x}_1$. Finalmente, añadiendo $\mathbf{M}$ a (3.12), obtenemos un nuevo tipo de autoatención enmascarada:

$$\mathbf{Y} = \text{Softmax}[\mathbf{Q}\mathbf{K}^T + \mathbf{M}]\mathbf{V} \quad (4.4)$$

$$\mathbf{Y}[\mathbf{X}] = \text{AtenciónMáscara}[\mathbf{Q}, \mathbf{K}, \mathbf{V}] \quad (4.5)$$

4.3.1.3. Muestreo

Una vez calculada la distribución de probabilidades para el siguiente *token*, necesitamos una estrategia para elegir cuál será el que extienda la secuencia. La opción más sencilla consiste en elegir siempre el *token* con la probabilidad más alta, pero esto convierte el modelo en un sistema determinista y no nos asegura que la secuencia completa generada sea globalmente la más probable. Otra opción sería elegir el siguiente *token* aleatoriamente, lo cual queda completamente descartado ya que el modelo generaría resultados sin sentido.

Lo ideal para encontrar la secuencia más probable sería aquella que hiciese máxima la probabilidad conjunta de toda la frase:

$$p(\mathbf{y}_1, \dots, \mathbf{y}_N) = \prod_{n=1}^N p(\mathbf{y}_n \mid \mathbf{y}_1, \dots, \mathbf{y}_{n-1}) \quad (4.6)$$

Sin embargo, esto es computacionalmente muy costoso, del orden $\mathcal{O}(k^N)$ donde, recordemos, k es el tamaño del vocabulario $\mathcal{V}$. Existen entonces varios métodos de muestreo de los cuales explicaremos los dos más usados: muestreo Top- K y muestreo Top- p .

El primero, muestreo Top- K , ordena todos los *tokens* del vocabulario de mayor a menor probabilidad y se queda únicamente con los K *tokens* más probables. Después, las probabilidades de estos K *tokens* se normalizan para que sumen 1 y se elige el siguiente realizando un muestreo aleatorio sobre este subconjunto. Una de las principales desventajas de este método es que K es fijo y puede haber situaciones en las que el modelo esté obligado a añadir al subconjunto *tokens* muy poco probables.

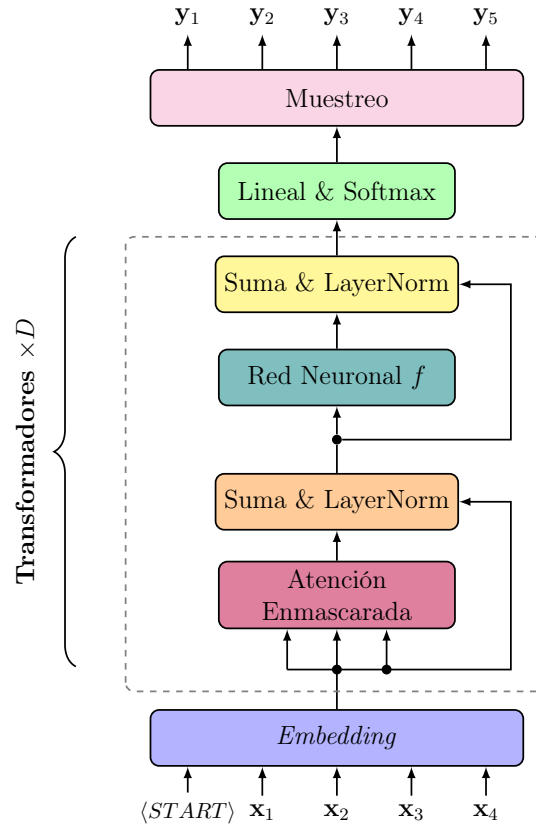


Figura 4.4: Arquitectura del decodificador. El decodificador recibe la entrada tokenizada y la convierte en vectores mediante la matriz de *embedding*. En la práctica, se apilan D bloques transformador, donde la salida de uno conforma la entrada del siguiente. Sobre la salida del último bloque se aplica una transformación lineal y la función Softmax para, tras el muestreo, obtener los *tokens* generados y_1, y_2, y_3, y_4, y_5 . El uso de la atención enmascarada permite procesar estas secuencias de datos en paralelo y optimizar el ajuste de los pesos, sin tener que calcular los vectores de salida secuencialmente.

El segundo método, muestreo Top- p , soluciona el problema del K fijo y elige dinámicamente la cantidad de *tokens* que se consideran en cada iteración. La idea principal de Top- p es seleccionar en cada paso el conjunto más pequeño de *tokens* más probables cuya probabilidad total sea mayor o igual al umbral especificado p . Este conjunto se llamará el núcleo $\mathcal{V}^p$ y matemáticamente se define como :

$$\sum_{x \in \mathcal{V}^p} P(x \mid x_{1:i-1}) \geq p \quad \text{y} \quad \forall S \subset \mathcal{V}^p, \sum_{x \in S} P(x \mid x_{1:i-1}) < p \quad (4.7)$$

Aunque una formulación equivalente es ordenar los *tokens* en orden descendente y añadir al núcleo aquellos cuya masa acumulada sea mayor o igual que p .

4.3.2. Codificador

Los codificadores son modelos cuyo propósito principal es construir una representación del texto capaz de encapsular la sintaxis y las relaciones semánticas entre los *tokens* que reciben como entrada. A nivel arquitectónico, un codificador está formado por múltiples transformadores apilados donde la salida de una capa sirve como entrada de la siguiente. Todo este proceso para aprender el lenguaje y resolver tareas se lleva a cabo en dos grandes fases: una de pre-entrenamiento y otra de ajuste fino (*fine tuning*).

En este contexto, el modelo basado en codificadores más disruptivo fue BERT (*Bidirectional Encoder Representations from Transformers*) pues mientras que los modelos de lenguaje anteriores procesaban la información de forma estrictamente unidireccional (de izquierda a derecha), BERT introdujo la capacidad de procesar el contexto en ambas direcciones simultáneamente aprovechando los mecanismos de autoatención del transformador. Por lo que en esta sección estudiaremos la arquitectura y el comportamiento de BERT para la clasificación de texto.

Una característica fundamental y particular de la arquitectura de BERT es cómo prepara la entrada: el primer token de cada secuencia es siempre un *token* especial de clasificación denotado como $\langle \text{CLS} \rangle$. Este *token* será el encargado de clasificar el texto de entrada.

4.3.2.1. Pre-entrenamiento

En la fase de pre-entrenamiento el modelo asimila las reglas semánticas y sintácticas del lenguaje mediante aprendizaje autosupervisado. Sin embargo, a diferencia del decodificador, que intenta predecir cuál es la siguiente palabra en una secuencia, el codificador se entrena para adivinar qué palabras faltan dentro de una frase.

Para lograrlo, el codificador recibe una secuencia donde un porcentaje de los *tokens* originales se han ocultado y sustituido por un *token* especial denominado $\langle \text{MASK} \rangle$. Por ejemplo, dada la frase “El gato está sobre la mesa”, si suponemos que se ocultan las palabras “gato” y “mesa”, la entrada al modelo sería:

$\langle \text{CLS} \rangle$ El $\langle \text{MASK} \rangle$ está sobre la $\langle \text{MASK} \rangle$

El objetivo del modelo es entonces reconstruir la frase prediciendo correctamente las palabras enmascaradas. En comparación con el decodificador, el codificador tiene acceso a la secuencia completa, lo que le permite prestar atención a todos los *tokens* que rodean a cada $\langle \text{MASK} \rangle$ (tanto a su izquierda como a su derecha). Al predecir las palabras faltantes apoyándose en este contexto bidireccional, el modelo interioriza de manera implícita el significado del lenguaje.

A nivel arquitectónico, el codificador está formado por múltiples transformadores apilados, donde la salida de uno sirve como entrada del siguiente. Si suponemos que el codificador recibe como entrada N *tokens* representados por los vectores $\mathbf{x}_1, \dots, \mathbf{x}_N$, la salida del último transformador será un conjunto de N vectores $\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_N$, los cuales conservan la misma dimensión D_i que los *embeddings* iniciales.

Para generar las predicciones finales, a esta matriz de salida $\tilde{\mathbf{X}}$ se le aplica una transformación lineal definida por la matriz de pesos $\mathbf{\Omega}_o$, seguida de una función de

activación Softmax (de manera análoga al decodificador):

$$\mathbf{Y} = \text{Softmax}(\tilde{\mathbf{X}}\mathbf{\Omega}_o) \quad (4.8)$$

La matriz $\mathbf{\Omega}_o$ tiene dimensiones $D_i \times k$, donde $k = |\mathcal{V}|$ representa el tamaño del vocabulario. Por lo tanto, la matriz resultante $\mathbf{Y}$ está compuesta por N filas (los vectores $\mathbf{y}_n$), cada una de dimensión k . Cada vector $\mathbf{y}_n$ representa una distribución de probabilidad sobre el vocabulario, indicando la probabilidad que tiene cada palabra de ocupar la posición n -ésima en la frase. Finalmente, durante la optimización del modelo, el cálculo del error solo tendrá en cuenta las distribuciones de probabilidad generadas en las posiciones que originalmente ocupaba el *token* $\langle \text{MASK} \rangle$. Destacar que durante esta fase de pre-entrenamiento el *token* $\langle \text{CLS} \rangle$ se omite y su salida $\mathbf{y}_0$ correspondiente también se omite.

4.3.2.2. Ajuste Fino (*Fine-Tuning*)

Una vez terminada la fase de pre-entrenamiento el modelo se ha convertido en un experto en las reglas del lenguaje, pero en el fondo solo sabe hacer una cosa: adivinar palabras ocultas. Para que nos sirva en el mundo real y resuelva problemas útiles, necesitamos pasarlo por una segunda etapa que llamamos ajuste fino (*fine tuning*). Básicamente, se trata de tomar este modelo generalista inicializado con los parámetros pre-entrenados y enseñarle una tarea específica en un entorno supervisado, es decir, con datos etiquetados.

En nuestro caso la tarea específica será clasificar el texto en C clases distintas. Es aquí entonces donde el *tokenspecial* $\langle \text{CLS} \rangle$ demuestra su utilidad. Durante el ajuste fino, eliminaremos la última capa lineal y la función Softmax y tomaremos el vector salida $\tilde{\mathbf{x}}_0$ del último transformador apilado correspondiente al primer *token* de entrada $\langle \text{CLS} \rangle$. Este vector actúa como la representación global de la frase pues en esta fase, a diferencia del pre-entrenamiento, el primer *token* $\langle \text{CLS} \rangle$ prestará atención al resto de *tokens* y servirá como vector que resume el significado global de la frase o secuencia de *tokens*. A este vector $\tilde{\mathbf{x}}_0$ le aplicaremos una matriz de pesos $\mathbf{\Omega}_o$ de dimensión $D_i \times C$ seguido de la función Softmax. De este modo, la salida del modelo se transforma en una distribución de probabilidad que indicará la probabilidad de que la secuencia de entrada pertenezca a cada una de las C clases posibles. En esta fase, el resto de vectores $\tilde{\mathbf{x}}_1 \dots \tilde{\mathbf{x}}_N$ los omitiremos donde N es el número de *tokens* que recibe como entrada.

Durante este entrenamiento, el modelo aprende a usar su nueva capa de clasificación para dar la respuesta correcta, a la vez que se afinan conjuntamente todos los parámetros originales acumulados en el pre-entrenamiento. Como ya partimos de una base excelente, estos reajustes son relativamente económicos computacionalmente y el modelo se adapta a la nueva tarea rapidísimo. Esta es la gran ventaja del enfoque de ajuste fino: nos permite alcanzar resultados punteros agregando un mínimo de parámetros específicos para la tarea, evitando tener que diseñar arquitecturas complejas desde cero para cada problema distinto.

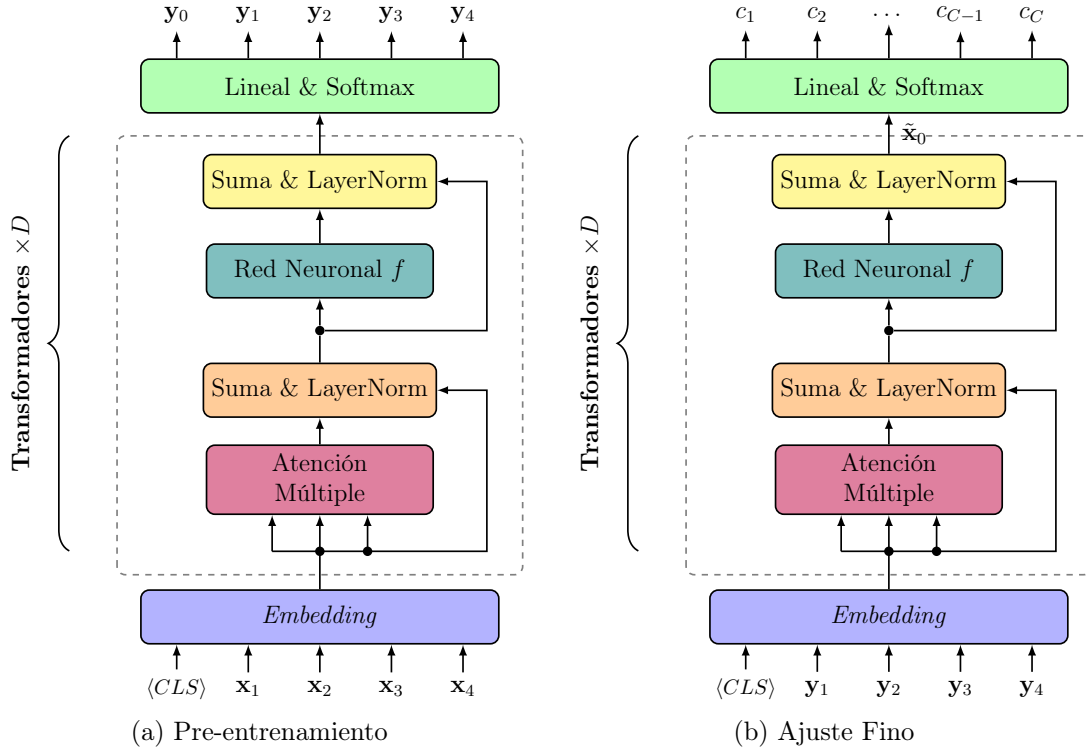


Figura 4.5: Arquitectura de los codificadores según la fase. En la figura a) la salida son probabilidades sobre el vocabulario. Si, por ejemplo, el *token* $\mathbf{x}_2$ estuviese oculto, el vector $\mathbf{y}_2$ se tendría en cuenta para el cálculo del error y posteriormente el ajuste de los pesos de la red. Además, los vectores $\langle CLS \rangle$ e $\mathbf{y}_0$ no se tienen en cuenta durante esta fase. En la figura b) solo consideramos la salida $\tilde{\mathbf{x}}_0$ del transformador correspondiente al *token* inicial $\langle CLS \rangle$. Sobre este vector $\tilde{\mathbf{x}}_0$ aplicamos la transformación lineal seguido de la función Softmax obteniendo una distribución de probabilidad sobre las distintas clases. En ambas arquitecturas no se modifica la atención y se trata del mismo transformador que en el capítulo 3.

4.3.3. Codificador-Decodificador

La arquitectura codificador-decodificador es un modelo híbrido que combina las fortalezas de los dos modelos anteriores. De hecho, en el artículo original *Attention is all you need*, el transformador se presentó como un codificador-decodificador. Sin embargo, en este trabajo se ha optado por presentar el transformador como una arquitectura general y, después, explicar sus variantes para facilitar la comprensión antes de unir las en una sola pieza.

El objetivo principal de codificador-decodificador es relacionar dos dominios distintos como ocurre, por ejemplo, en un traductor de inglés a español. En este escenario, el codificador se encarga de analizar y encapsular la semántica del inglés, mientras que el decodificador asume la tarea de generar la secuencia equivalente en español. Ambos no trabajan de manera aislada, sino que cooperan estrechamente a través de un mecanismo fundamental: la atención cruzada (*cross attention*).

4.3.3.1. Atención Cruzada (*cross attention*)

Supongamos que queremos entrenar un modelo capaz de traducir del inglés al español. El codificador recibe y procesa la frase original en inglés, y el decodificador va generando la traducción al español, palabra por palabra, utilizando su propio mecanismo de autatención enmascarada. Sin embargo, para que el decodificador pueda realizar una traducción fiel necesita tener acceso directo a la información que el codificador acaba de aprender o extraer. Es aquí donde entra en juego la atención cruzada.

La atención cruzada funciona de manera idéntica a la autoatención explicada en el capítulo 3, pero con una diferencia crucial en el origen de los datos: la clave y el valor provienen de la salida final del codificador y la consulta proviene del decodificador. De forma intuitiva es que en cada paso de la generación de texto el decodificador, a través de su consulta, le “pregunta” al codificador en qué partes exactas de la frase original en inglés debe concentrarse en ese preciso instante para extraer el significado adecuado, es decir, los valores, y así poder predecir la siguiente palabra en español. De este modo, la traducción avanza siempre guiada por el contexto de la frase original.

Finalmente, es importante destacar que el éxito de esta arquitectura reside en su entrenamiento mediante un enfoque supervisado, es decir, con un conjunto de datos etiquetados. Durante este proceso, el modelo recibe parejas de secuencias (por ejemplo, una frase en inglés y su traducción correspondiente en español) y mientras el codificador procesa la frase original completa, el decodificador se entrena para predecir la siguiente palabra de la traducción utilizando el contexto extraído por la atención cruzada. Al comparar la predicción del modelo con la palabra real de la etiqueta, se calcula un error que permite ajustar todos los parámetros de la red simultáneamente. Gracias a esta cooperación el sistema no solo aprende a traducir palabras sueltas, sino que desarrolla la capacidad de captar conceptos abstractos que pueden transferirse entre diferentes idiomas y dominios

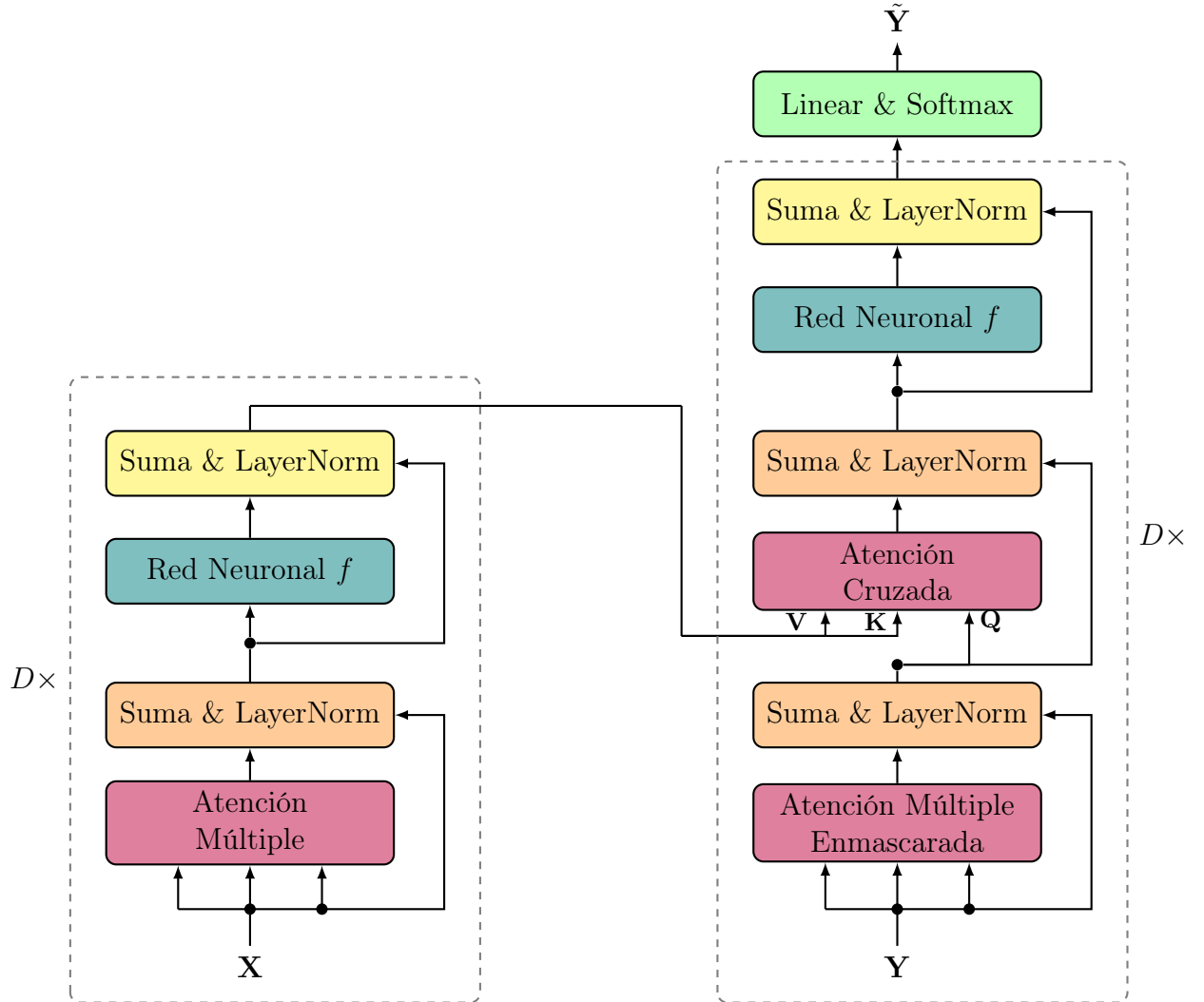


Figura 4.6: Arquitectura del codificador-decodificador. El bloque izquierdo representa el codificador y el bloque derecho el decodificador. Destacar que este codificador son D transformadores apilados donde a la última salida no se le aplica ninguna capa lineal ni de Softmax. Esto permite que la salida del codificador tenga la misma dimensión para, posteriormente, calcular las matrices $\mathbf{K}$ y $\mathbf{V}$ en la atención cruzada. En esta arquitectura, el decodificador tiene una nueva etapa, la atención cruzada, donde cada uno de los D decodificadores apilados recibe la salida del codificador. La salida $\tilde{\mathbf{Y}}$ representa las probabilidades por lo que sería necesario realizar un muestreo.

Capítulo 5

Implementación

Tras haber dedicado los capítulos anteriores a analizar detalladamente los fundamentos teóricos y las distintas variantes de la arquitectura del transformador, el siguiente paso lógico es su aplicación práctica. Además surge la curiosidad de ver cómo se comporta el modelo cuando se enfrenta a una tarea real y compleja. Por este motivo, el núcleo de este capítulo se centra en la implementación de un transformador de tipo codificador-decodificador, diseñado específicamente para traducir frases del inglés al español.

Este capítulo tiene como objetivo describir todo el proceso de desarrollo de forma estructurada. Para ello, se abordarán los detalles técnicos que han hecho posible el entrenamiento del modelo, empezando por la selección y limpieza del conjunto de datos empleado. También se detallarán los hiperparámetros que definen la estructura de la red y otros criterios de diseño que, aunque no se incluyeron en la parte teórica para no desviar la atención, resultan fundamentales para que el sistema funcione correctamente en la práctica.

En la actualidad, existen numerosas librerías que permiten importar modelos pre-entrenados o utilizar arquitecturas ya implementadas con apenas unas pocas líneas de código. Sin embargo, dado que el propósito de este trabajo es pedagógico y analítico, se ha optado por prescindir de estas herramientas de alto nivel y programar el traductor íntegramente desde cero utilizando Python y la librería científica PyTorch. Esta decisión permite profundizar en los mecanismos internos del sistema y comprender de primera mano todos los detalles técnicos que son fundamentales para el correcto funcionamiento del modelo.

Finalmente, para facilitar la reproducción del experimento y la transparencia del trabajo, el código fuente y la estructura del proyecto están disponibles en un repositorio público de [GitHub](#). Allí se incluye también todo lo necesario para configurar el entorno fácilmente en una sola línea de comando y probar el funcionamiento del modelo mediante una interfaz donde el usuario puede introducir frases en inglés de forma sencilla y observar la traducción generada por el sistema.

5.1. Conjunto de datos: OPUS100

Para el desarrollo de cualquier modelo de aprendizaje automático, el primer paso fundamental consiste en la obtención y preparación de un conjunto de datos inicial. En nuestro caso, al tratarse de un sistema de traducción de inglés a español, es imprescindible disponer de un corpus compuesto por pares de frases traducidas con precisión. Para este propósito, se ha seleccionado el conjunto de datos OPUS100 [8]: un corpus de traducción de inglés a 100 idiomas distintos¹. En nuestro caso, solo utilizaremos aquellas frases traducidas de inglés a español².

Este corpus se estructura en tres subconjuntos diferenciados que permiten gestionar las distintas fases del desarrollo:

- **Conjunto de entrenamiento:** 1.000.000 de pares de frases, destinados al ajuste de los parámetros y pesos de la red.
- **Conjunto de validación:** 2.000 pares de frases, utilizados para supervisar el rendimiento durante el entrenamiento y evitar el sobreajuste.
- **Conjunto de prueba:** 2.000 pares de frases, reservados para la evaluación final del modelo en un entorno no visto.

Destacar que, aunque a priori el tamaño del conjunto seleccionado pueda parecer considerable, su escala resulta modesta al compararse con los grandes sistemas de traducción desarrollados por entidades como Google. Estos modelos han sido entrenados con un volumen aproximado de 25.000 millones de pares de datos, tal y como se analiza en el artículo *Massively Multilingual Neural Machine Translation in the Wild: Findings and Challenges* [1]. Otro ejemplo sería el modelo de traducción del artículo *Attention is all you need* [9] donde se utilizó un conjunto de datos de 36 millones de pares de frases traducidas.

5.1.1. Filtrado y calidad de los datos

En el ámbito de la inteligencia artificial, la eficacia de la arquitectura depende también de la calidad de la información con la que se entrena. Por ello, se ha diseñado un proceso de filtrado para eliminar el ruido y las inconsistencias en el conjunto original:

1. **Limpieza de datos nulos:** Se han descartado todas las entradas que contengan valores nulos o donde alguna de las cadenas de texto (inglesa o española) esté vacía.
2. **Longitud mínima:** Se han eliminado aquellas traducciones en las que alguna de las frases posea menos de tres palabras, con el fin de asegurar un mínimo de carga semántica.

¹Al principio, durante el desarrollo del proyecto, se empleó otro conjunto de datos. No obstante, tras varias pruebas se observó que los datos eran de muy mala calidad pues las traducciones apenas tenían sentido y muchas de ellas tenían datos “basura”.

²Todo el código correspondiente al procesamiento y tratamiento del conjunto de datos se encuentra detallado en el archivo `notebooks/dataset.ipynb`.

Estadístico	L_{en}	L_{es}	r
Media	12,94	13,46	1,02
Desviación típica	13,21	14,84	0,24
Mínimo	3,00	3,00	0,50
25 %	5,00	5,00	0,83
50 %	8,00	8,00	1,00
75 %	16,00	16,00	1,17
90 %	28,00	31,00	1,33
95 %	38,00	43,00	1,45
Máximo	1737,00	1721,00	1,98

Tabla 5.1: Estadísticas descriptivas del corpus filtrado para las longitudes en inglés (L_{en}), español (L_{es}) y el ratio (r).

3. Detección de anomalías: Se han filtrado aquellos casos extraños donde la traducción al español resulta ser una copia exacta de la frase original en inglés.

Además, para garantizar la coherencia lingüística de los pares, hemos definido una métrica denominada *ratio* (r). Este valor se define como el cociente entre la longitud de la frase de salida y la de entrada:

$$r = \frac{L_{es}}{L_{en}} \quad (5.1)$$

donde L_{es} es la longitud de la frase en español y L_{en} es la longitud de la frase en inglés³. Es razonable esperar que las extensiones de las frases no difieran de manera excesiva. Un ratio superior a 2 indicaría que la traducción emplea más del doble de palabras que el original, lo cual resulta anómalo en nuestro dominio. Por otro lado, un valor inferior a 0,5 sugeriría una pérdida significativa de información o una traducción incompleta. Por tanto, se ha optado por conservar únicamente aquellas traducciones cuyo ratio se encuentre dentro del intervalo $(\frac{1}{2}, 2)$.

Finalmente, y tras aplicar el filtrado, el conjunto de entrenamiento se reduce a 759.856 traducciones, es decir, se han descartado un total de 240.144 traducciones del conjunto inicial.

5.1.2. Estructura de los datos

Tras descartar los datos de baja calidad, conviene estudiar la estructura y las características de los datos que componen el corpus. Este análisis permite comprender la distribución de las longitudes y el ratio de las secuencias. Para ello, se han calculado la media, la desviación típica y los percentiles del 25 %, 50 %, 75 %, 90 % y 95 % sobre la longitud de las frases y su ratio, obteniendo los resultados que se muestran en la Tabla 5.1. Asimismo, para obtener una perspectiva más intuitiva de cómo se organizan o reparten los datos, se puede observar la distribución de estas mediante los histogramas presentados en la figura 5.1.

³Aquí, longitud de la frase hace referencia al número de palabras que componen la frase.

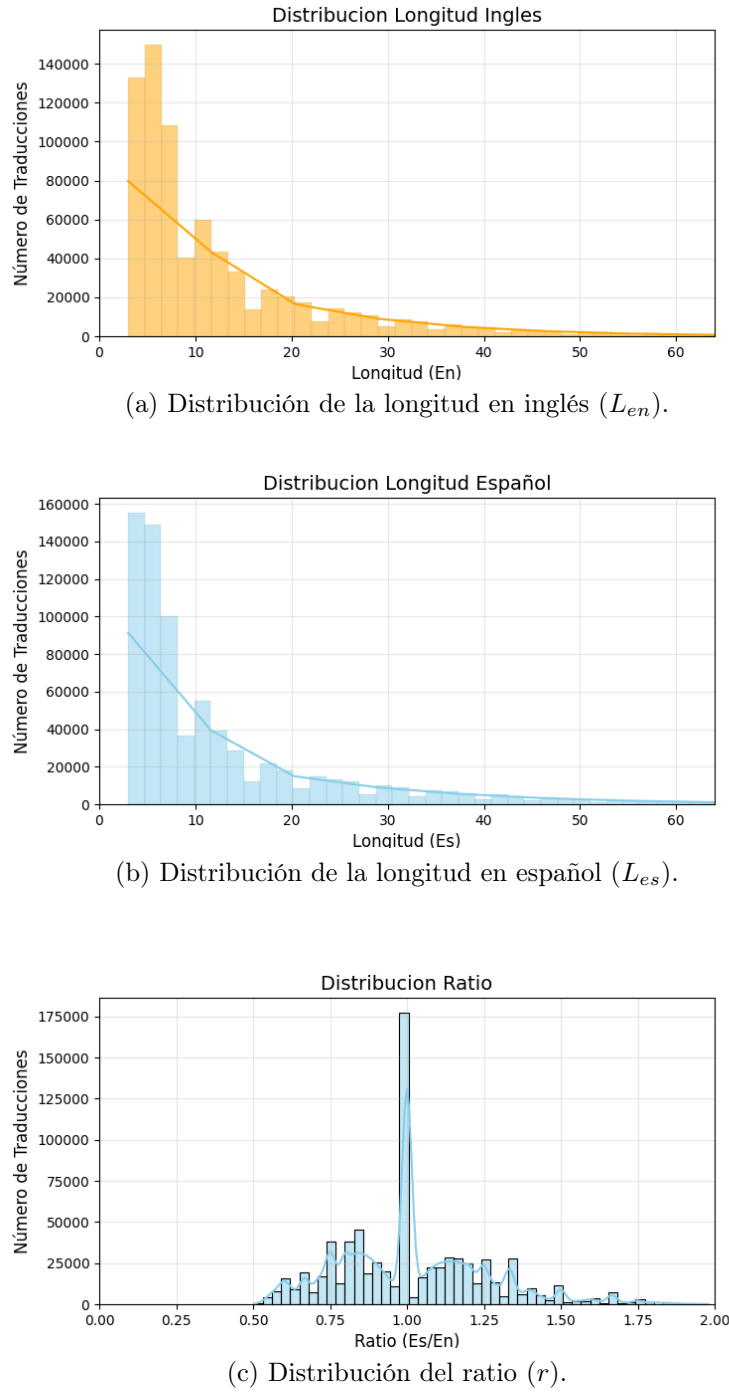


Figura 5.1: Histogramas de frecuencias para las longitudes de las frases en ambos idiomas y la distribución del ratio de traducción en el corpus filtrado.

Se observa que, aunque el rango de longitudes es amplio, el 90 % de las frases en español poseen 31 palabras o menos. El ratio medio de 1,02 garantiza un conjunto de datos equilibrado para el proceso de aprendizaje.

Otro punto de interés reside en los valores máximos registrados, que ascienden a 1.737 palabras en inglés y 1.721 en español. Se observa una diferencia sustancial

Idioma	Total de palabras únicas
Inglés	152.068
Español	208.369

Tabla 5.2: Riqueza léxica del corpus expresada en número de vocablos distintos.

Rango	Palabra (EN)	Frecuencia (EN)	Palabra (ES)	Frecuencia (ES)
1	the	532.625	de	582.017
2	of	268.586	la	331.605
3	to	249.572	que	266.058
4	and	222.554	el	237.683
5	you	182.139	en	230.591
6	i	179.156	y	218.430
7	a	174.868	a	205.047
8	in	165.885	los	142.553
9	that	114.410	no	140.768
10	it	104.917	un	101.677

Tabla 5.3: Clasificación de las diez palabras más frecuentes identificadas en el corpus.

con la tendencia mayoritaria, ya que el 95 % de las frases contiene 43 palabras o menos. Por este motivo, y en previsión del número N de *tokens* que el transformador será capaz de procesar y que se detallará más adelante, se ha optado por descartar aquellas frases cuya longitud exceda las 64 palabras. Se han eliminado 10.753 pares de frases y, finalmente, el corpus resultante está formado por 749.103 traducciones.

Para completar el estudio descriptivo, resulta útil e interesante evaluar la amplitud del vocabulario en ambos idiomas, es decir, cuantificar la cantidad de palabras distintas que el modelo encontrará durante el entrenamiento. En la Tabla 5.2 se detallan los resultados obtenidos para cada lengua donde se observa que el español presenta un número significativamente mayor de palabras únicas en comparación con el inglés. Sin embargo, esto no es un problema y, en cierto sentido, es razonable esa diferencia. Uno de los motivos es, por ejemplo, la forma de conjugar verbos: en inglés el verbo no suele cambiar mucho a diferencia del español que tiene decenas de formas distintas según el tiempo, la persona y el número. Otro motivo es que los adjetivos en inglés son invariables: la palabra “red” sirve para describir cualquier objeto sin importar género o cantidad; en cambio, en español esa misma palabra se divide en cuatro formas distintas “rojo”, “roja”, “rojos” y “rojas”.

También se ha realizado un análisis de las frecuencias de aparición para identificar las palabras predominantes en el corpus. En la Tabla 5.3 se recogen las diez palabras más frecuentes para cada idioma.

Los resultados confirman que las palabras más utilizadas en ambos idiomas pertenecen a la categoría de palabras funcionales (artículos, preposiciones y conjunciones). Estos términos, aunque carecen de un peso semántico específico por sí solos, son los encargados de articular la estructura sintáctica de las oraciones. Destaca la frecuencia de la palabra “the” en inglés y la preposición “de” en español, encabezando sus respectivos listados con una frecuencia considerablemente superior al resto de

términos.

5.2. Vocabulario

Una vez tenemos los datos con los que entrenar el modelo, el siguiente paso crítico en la implementación es el diseño del vocabulario. Inicialmente se planteó el uso de un vocabulario ya creado como el empleado en la arquitectura GPT2; sin embargo, se concluyó que dicha aproximación resultaría contraproducente para los fines de este trabajo. Los modelos de propósito general se entrenan con corpus masivos que incluyen desde literatura hasta código de programación, lo que genera una distribución de *tokens* muy alejada de nuestro conjunto de traducción. Es por ello que emplear un vocabulario ajeno podría provocar una fragmentación excesiva de las palabras comunes de nuestro dominio en múltiples *tokens*. Esto no solo incrementaría la longitud de las secuencias de entrada, sino que perjudicaría al mecanismo de autoatención del modelo: sería, en cierto modo, como considerar un vocabulario compuesto únicamente por caracteres.

Teniendo todo esto en cuenta, se ha optado por construir un vocabulario $\mathcal{V}$ propio mediante la aplicación del algoritmo BPE (algoritmo 1) sobre el conjunto de entrenamiento, es decir, sobre la unión de las frases en inglés y sus correspondientes traducciones al español. Inicialmente, $\mathcal{V}$ se compone de los caracteres individuales de ambos alfabetos. No obstante, para garantizar la calidad del vocabulario resultante, se debieron definir ciertas precondiciones en el proceso de construcción.

El primer problema al aplicar el algoritmo BPE directamente sobre el texto fue la generación de *tokens* demasiado largos, que llegaban a ser frases casi completas. Esto ocurría porque, al repetir el proceso, las secuencias más comunes acababan siendo palabras enteras ya unidas entre sí, lo que restaba flexibilidad al modelo a la hora de traducir ciertas frases. Para evitarlo, en lugar de procesar todo el texto de seguido, decidimos aplicar el algoritmo palabra por palabra. De esta forma, si llamamos $\mathcal{T}$ al conjunto de todas las frases del corpus, el conjunto $\mathcal{S}$ sobre el que aplicamos BPE pasa a ser la unión de las palabras individuales que forman dichas frases:

$$\mathcal{S} = \bigcup_{t_i \in \mathcal{T}} \{w : w \in t_i\} \quad (5.2)$$

donde w denota una palabra perteneciente a una frase t . Con esta restricción, nos aseguramos de que los caracteres solo se unan dentro de una misma palabra y evitamos que se formen *tokens* que sean frases enteras.

Sin embargo, al separar el texto en palabras individuales, apareció otro problema: la falta de espacios. El algoritmo perdía la conexión entre palabras y el modelo generaba frases bien escritas pero totalmente pegadas, como “Elgatoestásobrelamesa”. Para solucionarlo, modificamos la segmentación inicial añadiendo el carácter especial “.” al principio de cada palabra para representar el espacio en blanco. Al meter este símbolo en el vocabulario desde el principio, el BPE pudo integrarlo en los *tokens* más comunes. Por ejemplo, la frase “El gato” se procesa como “El” y “.gato”. Si el algoritmo decide que “ga” y “to” aparecen mucho, el vocabulario final incluirá los *tokens* “.ga” y “.to”. De este modo, el modelo reconstruye la palabra sabiendo que el

ID	Token	ID	Token
283	·the	316	·que
311	ing	348	ción
330	·and	412	·para
433	·with	490	ión
510	ould	663	·está
795	·would	1754	miento
1051	·people	2086	·nuestra
1105	·new	3694	·gobierno

Figura 5.2: Muestra representativa de tokens extraídos del vocabulario. El punto de multiplicación (·) representa un espacio en blanco inicial, indicando que el token marca el comienzo de una palabra.

primer fragmento indica el inicio de una palabra nueva tras un espacio. Esta técnica permite que el resultado final sea legible y que el transformador pueda descomponer palabras complejas en piezas con significado propio.

Finalmente, tras aplicar estos ajustes, el resultado es un vocabulario $\mathcal{V}$ de 16.000 *tokens* que mezcla raíces, prefijos, sufijos y/o palabras enteras de ambos idiomas. En la figura 5.2 se puede ver una pequeña muestra de los *tokens* que forman parte de este vocabulario final que utilizará el modelo tanto para comprender las frases originales en inglés como para realizar las correspondientes traducciones al español.

5.3. Transformador

Con el vocabulario listo y los datos procesados, el siguiente paso es configurar la arquitectura neuronal. Para este traductor, hemos seguido el diseño de codificador-decodificador propuesto en el artículo original *Attention Is All You Need*. Aunque la estructura general es la misma que explicamos en la sección 4.3.3, hay un detalle fundamental en el diagrama original (ver figura 5.3) que todavía no hemos analizado: la codificación posicional (*positional encoding*). Decidimos posponer su explicación para no sobrecargar los capítulos teóricos anteriores, pero es una pieza clave para que el sistema funcione en la práctica⁴.

5.3.1. Codificación posicional (*Positional Encoding*)

Si analizamos cómo funciona el mecanismo de autoatención, veremos que carece por completo de una noción de orden secuencial pues al procesar todos los *tokens* de forma paralela, el modelo es, por definición, invariante ante permutaciones. Sin embargo, en el lenguaje natural, el orden de las palabras es esencial para determinar el significado de la frase. Por ejemplo, no posee el mismo significado la frase “*El perro mordió al hombre*” que “*El hombre mordió al perro*”, a pesar de contener exactamente las mismas palabras.

⁴Todo el código correspondiente al modelo se encuentra en la carpeta `src`. La arquitectura implementada y su configuración están en `src/transformer.py`.

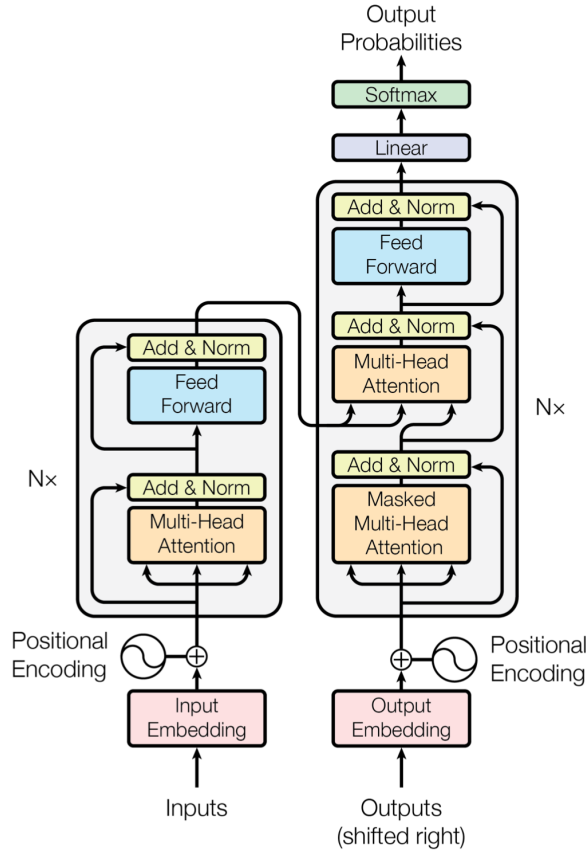


Figura 5.3: Imagen del artículo original donde se muestra la arquitectura del transformador.

Para integrar esta información de posición sin renunciar a las ventajas del procesamiento en paralelo, introducimos la codificación posicional (*positional encoding*). El objetivo es asociar un vector de posición $\mathbf{r}_i$ a cada *token* de entrada $\mathbf{x}_i$. Para ello sumaremos directamente el vector de posición al vector del *token*:

$$\tilde{\mathbf{x}}_i = \mathbf{x}_i + \mathbf{r}_i \quad (5.3)$$

obteniendo así un nuevo vector $\tilde{\mathbf{x}}_i$ que codifica tanto la información del *token* como su posición. Obviamente, para que esta operación sea posible es necesario que los vectores $\mathbf{r}_i$ tengan la misma dimensión D_i que los *tokens* $\mathbf{x}_n$. Existen diversas maneras de construir estos vectores $\{\mathbf{r}_i\}$. Una de las más conocidas es la basada en funciones sinusoidales de diferentes frecuencias, la cual permite que el modelo extrapole relaciones a secuencias de longitudes no vistas previamente (ver [2, Cap 12] para más detalles). No obstante, para nuestra implementación hemos optado por el enfoque de codificaciones posicionales aprendidas donde definiremos una matriz de pesos posicionales, $\Omega_{\mathbf{r}}$ de dimensión $D_i \times N$, donde cada columna representa un vector de posición específico para cada una de las N posiciones posibles:

$$\Omega_{\mathbf{r}} = \begin{bmatrix} | & | & \dots & | \\ \mathbf{r}_1 & \mathbf{r}_2 & \dots & \mathbf{r}_N \\ | & | & \dots & | \end{bmatrix} \quad (5.4)$$

Esta matriz irá ajustando sus valores durante el entrenamiento y, en consecuencia, el modelo aprenderá a determinar las posiciones de los *tokens* que recibe como entrada.

5.3.2. Hiperparámetros

Para ajustar el modelo a las características del corpus OPUS100 y a los recursos computacionales disponibles, se han definido los hiperparámetros que determinan la capacidad de aprendizaje y la complejidad estructural de la red:

- **Dimensión de los *tokens* (D_i):** 256. Representa la dimensionalidad de los *tokens* que procesa el transformador o, equivalentemente, la dimensión de los *embeddings*.
- **Número de codificadores apilados:** 4.
- **Número de decodificadores apilados:** 4.
- **Capas de autoatención (H):** 8. Número de mecanismos de atención que trabajan simultáneamente en paralelo.
- **Número máximo de tokens (N):** 128. Define el número máximo de *tokens* en los que se puede dividir una frase.
- **Cardinal del vocabulario (k):** 16.000. Tamaño total del diccionario de *tokens* compartidos entre ambos idiomas.

Inicialmente se intentó aplicar la misma configuración que el artículo original [9] donde se utilizaba la siguiente configuración:

- **Dimensión de los rasgos (D_i):** 512.
- **Bloques transformadores apilados (D):** 6.
- **Capas de autoatención (H):** 8.
- **Número máximo de tokens (N):** 512.
- **Cardinal del vocabulario (k):** 37.000.

Sin embargo, y teniendo en cuenta que los recursos son limitados, se decidió reducirlos para que el proceso de entrenamiento no se alargase mucho. Además de que previamente se realizaron varias pruebas de entrenamiento donde se estimó el tiempo que tardaría en entrenar.

5.3.3. Entrenamiento

Definidos los hiperparámetros pasamos a la fase de entrenamiento. Comentaremos detalles técnicos de esta fase siguiendo un poco la estructura del artículo original⁵.

⁵Todo el código correspondiente al entrenamiento se encuentra detallado en el archivo `src/train.py` y la ejecución del mismo en `notebooks/training_colab.ipynb`.

5.3.3.1. Hardware

Se utilizó una tarjeta gráfica (GPU) NVIDIA A100 a través de la plataforma Google Colab y se entrenó durante 45 épocas⁶ dividiendo el conjunto total en lotes (*batches*) de 128 traducciones. Cada época tardaba alrededor de unos 13 minutos y el entrenamiento total fueron en total 9 horas aproximadamente.

5.3.3.2. Criterio de Error y Función de Pérdida

En este modelo se ha empleado la entropía cruzada (*Cross Entropy*), ya que resulta la métrica óptima para abordar problemas de clasificación multiclase. Matemáticamente, para un conjunto de clases C , la entropía cruzada entre la distribución real y y la predicción del modelo p se define como:

$$L = - \sum_{c=1}^{|C|} y_c \log(p_c)$$

En nuestra implementación, $C = \mathcal{V}$ siendo $\mathcal{V}$ el vocabulario del modelo y donde cada *token* posible se trata como una categoría independiente. Se utiliza esta métrica porque penaliza de forma logarítmica las predicciones incorrectas, lo que acelera la convergencia y asegura que el modelo asigne la mayor probabilidad posible a la traducción exacta. En la figura 5.4 se puede ver la evolución de la función de pérdida L a lo largo del entrenamiento.

5.3.3.3. Optimización Mediante AdamW

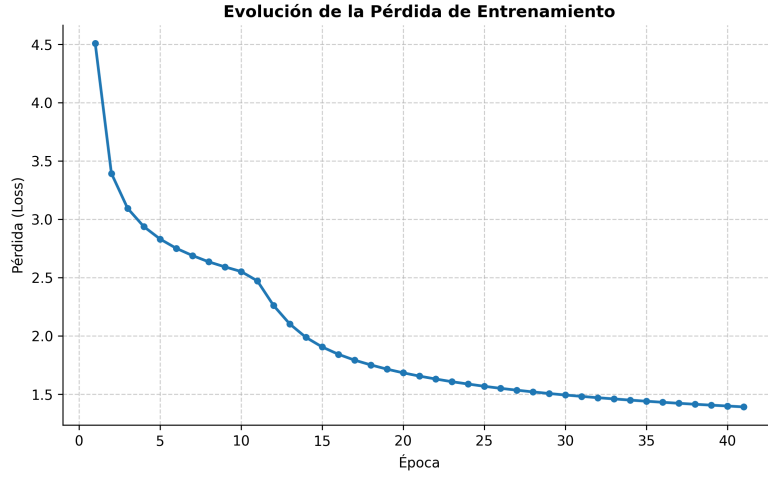
La actualización de los parámetros de la red se realiza mediante el algoritmo *AdamW*, una extensión del descenso de gradiente estocástico que incorpora momentos de primer y segundo orden para adaptar la tasa de aprendizaje de forma individual para cada peso. Matemáticamente, este optimizador mejora la generalización del modelo al desacoplar el decaimiento de los pesos (*weight decay*) de la actualización del gradiente. Se han mantenido los valores de diseño originales: $\beta_1 = 0,9$, $\beta_2 = 0,98$ y un factor de estabilidad $\epsilon = 10^{-9}$ para evitar divisiones por cero en el cálculo de las medias móviles.

5.3.4. Planificación de la tasa de aprendizaje

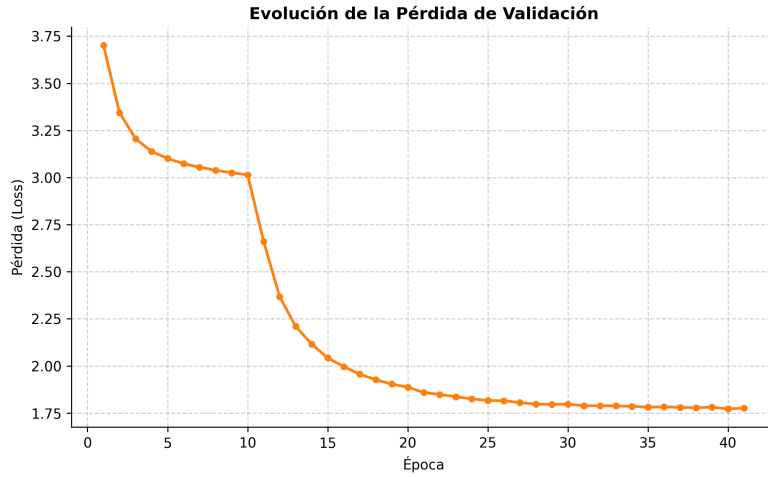
El entrenamiento de la arquitectura del transformador es altamente sensible a la selección de la tasa de aprendizaje (η). Por este motivo, en lugar de emplear un valor constante durante todo el proceso, se utiliza un planificador dinámico que ajusta este hiperparámetro en función del paso actual⁷. Matemáticamente, el comportamiento

⁶Una época es una pasada completa de todo el conjunto de datos de entrenamiento a través de la red neuronal. En otras palabras, se completa una época cuando el modelo ha “visto” y procesado cada uno de los ejemplos de entrenamiento exactamente una vez para ajustar sus pesos y aprender de los errores cometidos.

⁷Un paso se define cómo cada iteración de actualización de los parámetros del modelo tras procesar un lote (*batch*) de datos.



(a) Evolución de la pérdida de entrenamiento.



(b) Evolución de la pérdida de validación.

Figura 5.4: Evolución de la función de pérdida L calculada como el promedio sobre los respectivos conjuntos. El descenso simultáneo en ambas curvas, considerando que los conjuntos de entrenamiento y validación son disjuntos, es un indicador de que el modelo está aprendiendo y no simplemente memorizando los datos. En las etapas finales de la curva b) se intuye que el modelo converge y que continuar el entrenamiento no mejorará significativamente el rendimiento. Es por ello que se decidió parar el entrenamiento, aunque inicialmente el planteamiento era entrenar durante 50 épocas.

de η se rige por la siguiente expresión:

$$\eta = \frac{1}{\sqrt{D_i}} \cdot \min \left(\frac{1}{\sqrt{\text{paso}_{\text{actual}}}}, \text{paso}_{\text{actual}} \cdot \text{calentamiento}^{-1,5} \right) \quad (5.5)$$

donde D_i , recordemos, representa la dimensión de los vectores $\mathbf{x}_n$ y *calentamiento* es un valor predefinido que marca la duración de la fase inicial que, en nuestro caso, es 4.000. Esta función permite que el entrenamiento transcurra a través de dos etapas

claramente diferenciadas:

- **Fase de calentamiento:** Durante los primeros 4000 pasos, el segundo término de la ecuación domina el cálculo, lo que provoca que la tasa de aprendizaje aumente de forma lineal desde cero. Este incremento progresivo es fundamental para la estabilidad del modelo en sus fases iniciales, ya que evita que los gradientes inestables, propios de unos pesos todavía aleatorios, provoquen que el entrenamiento diverja prematuramente.
- **Fase de decaimiento:** Una vez que el *paso_actual* supera el umbral de *calentamiento*, el primer término de la función se vuelve dominante. A partir de este punto, la tasa de aprendizaje comienza a reducirse de forma proporcional a la raíz cuadrada inversa del paso. Este descenso paulatino permite que el modelo realice ajustes cada vez más sutiles y precisos, facilitando una convergencia óptima hacia el mínimo de la función de pérdida.

5.3.5. Metodología de trabajo y experiencia personal

La implementación del modelo se ha realizado íntegramente en PyTorch. Aunque ya contaba con experiencia previa en el desarrollo de redes neuronales, especialmente con arquitecturas convolucionales, este proyecto supuso un reto técnico distinto. La mayor dificultad no reside tanto en programar la lógica teórica, sino en gestionar correctamente el flujo y las dimensiones de los tensores a lo largo de toda la red. En PyTorch, el rendimiento depende de aprovechar el paralelismo de las funciones matriciales y evitar bucles secuenciales, lo que obliga a tener un control absoluto sobre cómo cambia la forma de los datos en cada capa.

Antes de comenzar a escribir el código, realicé una labor de investigación para estudiar otras implementaciones existentes. Dado que internet ofrece una cantidad ingente de ejemplos, fue necesario hacer un filtrado de calidad para seleccionar referencias fiables. Como base principal, utilicé el trabajo de Andrej Karpathy [4], una de las figuras más influyentes en el campo de la inteligencia artificial, exdirector de IA en Tesla y cofundador de OpenAI. Su enfoque educativo para construir transformadores desde cero permitió asentar las bases de este traductor. También fue fundamental el libro interactivo *Dive into Deep Learning* [10], un recurso desarrollado por científicos de Amazon y diversas universidades que destaca por conectar de forma brillante la teoría matemática con su aplicación práctica en Python.

En cuanto al uso de herramientas de apoyo, utilicé modelos de lenguaje como Gemini y Claude. Sin embargo, no se emplearon para generar el código de forma automática, sino como una fuente de documentación dinámica: en caso de duda sobre el comportamiento de alguna función específica de PyTorch o sus argumentos, estos modelos permitían una consulta rápida y contrastada. Además, Claude resultó de gran utilidad para construir pequeños tests de prueba para cada componente de la red, asegurando que cada módulo funcionase correctamente antes de integrarlo en la estructura global.

Finalmente, debido al enorme coste computacional que implica entrenar un transformador, fue necesario recurrir a hardware especializado. Por su facilidad de uso, opté por la versión de pago de Google Colab, una plataforma basada en la nube que

Inglés (Original)	Español (Traducción)
I live in a small house.	Vivo en un pequeño hogar.
My name is John and I am a student.	Mi nombre será John y yo, y soy estudiante.
Do you like red apples?	- ¿Te gustan las manzanas rojas?
She works in a big hospital.	Trabaja en un gran hospital.
The book is on the table.	El libro se encuentra en la mesa.
We play football every weekend.	Juguemos el fútbol todos los fines de semana.
He is very happy today.	Hoy está muy contenta.
They have a fast car.	Tienen un auto rápido.
What time is it now?	¿Qué hora es ahora?
I want to drink cold water.	Quiero beber agua fría.

Tabla 5.4: Evaluación cualitativa de traducción - Nivel A1.

permite el acceso a tarjetas gráficas (GPU) de alto rendimiento. Para este trabajo, se adquirió un paquete de 100 unidades de computación que equivalen, aproximadamente, a unas 20 horas de uso. Aproximadamente la mitad se utilizó en las fases iniciales de pruebas y corrección de errores, mientras que las últimas 50 unidades se utilizaron para el entrenamiento final una vez ajustados todos los hiperparámetros, el dataset y el vocabulario. Esta gestión eficiente de los recursos fue clave, ya que en modelos de este tipo, optimizar la manipulación de los tensores es vital para no desperdiciar tiempo de computación y lograr que el sistema aprenda dentro de los límites disponibles.

5.3.6. Resultados

Aunque existen métricas estandarizadas para evaluar el rendimiento de los modelos de traducción automática, en este proyecto se ha optado por realizar una evaluación cualitativa de diseño propio adaptada a nuestro enfoque unidireccional. El objetivo de esta prueba no es arrojar métricas con significancia estadística o rigor científico a gran escala, sino proporcionar una muestra representativa y empírica de cómo se comporta el modelo en la práctica.

Para ello, se ha confeccionado un conjunto de pruebas compuesto por un total de 60 frases, distribuidas equitativamente (10 frases por categoría) entre los seis niveles de competencia lingüística del Marco Común Europeo de Referencia para las lenguas (A1, A2, B1, B2, C1 y C2). Esta estrategia permite observar, de manera exploratoria e intuitiva, la capacidad de la red para gestionar diferentes grados de complejidad gramatical y riqueza de vocabulario.

Los resultados de esta prueba (ver tablas 5.4, 5.5, 5.6, 5.7, 5.8 y 5.9) muestran de forma muy clara cómo se comporta el modelo: es capaz de defenderse con el inglés cotidiano, pero sufre notablemente cuando el texto se vuelve complejo.

En los niveles básicos (A1 y A2), se observa que la red ha aprendido bien la estructura general de las frases. Es capaz de traducir oraciones directas sin proble-

Inglés (Original)	Español (Traducción)
I went to the cinema with my friends yesterday.	Fui al cine con mis amigos ayer.
She is going to visit her grandmother next week.	Ella va a visitar a su abuela la próxima semana.
We didn't buy any bread at the supermarket.	No compramos pan por el supermercado.
Can you help me with my homework, please?	¿Puedes ayudarme con mis tareas?
He is taller than his older brother.	Es más alto que su hermano mayor.
I enjoy listening to music when I walk to school.	Yo disfruto escuchar música cuando yo voy a las clases.
The weather was very cold and it rained all day.	El tiempo era frío, la orejó todo el día.
You must wear a uniform to work.	Debe llevar un uniforme al trabajo.
Have you ever been to Paris?	¿Has estado alguna vez en París?
They are watching a funny movie on television.	Están mirando una película graciosa en televisión.

Tabla 5.5: Evaluación cualitativa de traducción - Nivel A2.

ma, como al pasar de “I live in a small house” a “Vivo en un pequeño hogar”. Sin embargo, ya se aprecian descuidos con el género y el número; por ejemplo, traduce el masculino “He is very happy” como “Hoy está muy contenta” o se refiere al museo como “cerrada”. También aparece la primera palabra inventada al traducir que llovió todo el día usando el verbo “orejó”.

A medida que pasamos a los niveles intermedios (B1 y B2), la dificultad aumenta por el uso de tiempos verbales más complejos y frases hechas. Aquí el modelo peca de traducir literalmente palabra por palabra. Esto hace que fracase al traducir palabra por palabra frases hechas como “see eye to eye” traducida a “no veo muy ojos”, cuando su significado real es “estar de acuerdo”. También demuestra falta de contexto al traducir “play the piano” como “jugaba por el piano” en lugar de tocar, y se pierde por completo cuando la estructura gramatical se invierte, como ocurre con “Had I known about...”, perdiendo el sentido de la frase original.

Finalmente, en los niveles avanzados (C1 y C2), el modelo sencillamente colapsa. Al encontrarse con vocabulario muy específico o poco común que apenas habrá visto en su fase de entrenamiento, empieza a alucinar texto. Por un lado, se inventa palabras que suenan parecido al español pero que no existen, como “recortizado”, “dismo” o “bacona”. Por otro lado, intenta adivinar términos complejos basándose en sus primeras letras, lo que provoca el fallo más llamativo de toda la prueba: traducir “circumvent” (esquivar o eludir) por “circuncidar”, lo que arrastra al resto de la oración a no tener ningún sentido.

En resumen, la prueba demuestra que el traductor funciona correctamente para un uso básico y literal. Sin embargo, para alcanzar una competencia lingüística de nivel nativo, sería necesario ampliar significativamente el conjunto de datos de en-

Inglés (Original)	Español (Traducción)
If it rains tomorrow, we will probably stay at home and watch a movie.	Si sucede mañana, probablemente quedaremos en casa y veremos una película.
I have been learning English for three years, but I still make mistakes.	He estado aprendiendo inglés durante tres años, pero todavía me equivoco.
She told me that she had already finished the report before the meeting.	Me dijo que ya había terminado su reporte antes del encuentro.
The museum, which was built in the 19th century, is currently closed.	El museo, construido en el siglo XIX, está actualmente cerrada.
I used to play the piano when I was younger, but I don't have time anymore.	Antes jugaba por el piano cuando era más joven, pero ya no tengo tiempo.
You shouldn't drink so much coffee because it might keep you awake.	No deberías beber mucho café porque podría mantenerte despierto.
We are looking forward to spending our holidays at the beach this summer.	Estamos deseando estar en nuestra fiesta en la playa en este verano.
The thieves were caught by the police while they were trying to escape.	Los ladrones fueron capturados por la policía mientras estaban tratando de escapar.
Despite the heavy traffic, we managed to arrive at the airport on time.	A pesar del tráfico pesado, conseguimos llegar al aeropuerto a tiempo.
Do you mind if I open the window to let some fresh air in?	¿Te importa si abro las ventanas para dejar un poco de aire fresco?

Tabla 5.6: Evaluación cualitativa de traducción - Nivel B1.

trenamiento, permitiendo a la red exponerse a una mayor variedad de estructuras complejas y vocabulario técnico. También modificar la arquitectura hacia un modelo de traducción bidireccional podría mejorar los resultados. Pues a diferencia del modelo actual, que procesa la información de unidireccional, un modelo bidireccional analiza la oración completa en ambos sentidos antes de traducirla, es decir, el codificador recibe y codifica ambas frases antes de traducirla. Esto permite que el mecanismo de atención capture el contexto global, entendiendo cómo el final de una frase puede cambiar el sentido del principio. Esta visión de conjunto es lo que permitiría al modelo identificar frases hechas o dobles sentidos, evitando traducciones literales erróneas y resolviendo ambigüedades que solo se aclaran al leer la oración en su totalidad.

5.4. Detalles de implementación

Existen ciertos aspectos técnicos que, aunque fundamentales para el funcionamiento del modelo, se han reservado para este capítulo con el objetivo de agilizar la exposición teórica de las secciones anteriores.

Inglés (Original)	Español (Traducción)
The new policy is expected to bring about significant changes in the economy.	Se espera que la nueva política lleve a cabo los cambios significativos en la economía.
Had I known about the severe weather, I would have postponed my trip.	Me hubiera pasado por lo grave que tenía que haber puesto en mi viaje.
She is highly regarded by her colleagues for her outstanding leadership skills.	Es muy considerado por sus colegas por sus habilidades de liderazgo.
It goes without saying that hard work and dedication are crucial for success.	Va sin decir que la trabajadora de los más difícil es un logro crucial para el éxito.
The company's profits have soared recently, largely due to a marketing campaign.	Los beneficios de la empresa han sufrido recientemente, principalmente debido a su campaña de comercialización.
Not only did they fail to meet the deadline, but they also refused to apologize.	No solamente rechazaron el plazo, sino que también se negaron a disculparse.
By the time we graduate next year, we will have completed all the coursework.	Para el momento de graduarse el año próximo, tendremos todo el transcurso.
Taking all factors into consideration, the board has decided to approve the merger.	El Consejo ha decidido aprobar los factores de la fusión.
I'm afraid I don't quite see eye to eye with you on this particular issue.	Me temo que no veo muy ojos en este asunto particular.
The government has pledged to crack down on illegal dumping of hazardous waste.	El Gobierno ha prometido caer sobre el vertido ilícito de residuos peligrosos.

Tabla 5.7: Evaluación cualitativa de traducción - Nivel B2.

5.4.1. Escalado del producto escalar

El primer detalle de implementación se centra en la ecuación de atención descrita en el capítulo 3 (ver 3.12). En el artículo original, la función de atención se redefine mediante la introducción de un factor de escalado $1/\sqrt{D_i}$:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax} \left(\frac{\mathbf{QK}^T}{\sqrt{D_i}} \right) \mathbf{V} \quad (5.6)$$

Donde D_i representa la dimensionalidad de los vectores de entrada $\mathbf{x}_n$. Esta modificación responde a una necesidad práctica detectada por los autores, pues observaron que para valores elevados de D_i , el producto escalar tiende a crecer excesivamente en magnitud. Esto desplaza la función *Softmax* hacia regiones de saturación donde el gradiente es extremadamente pequeño, lo que dificulta el entrenamiento del modelo y al dividir por $\sqrt{D_i}$, se contrarresta este efecto de magnitud, manteniendo los valores en un rango que permite que los gradientes fluyan correctamente sin que el mecanismo de atención se vea afectado o pierda capacidad de aprendizaje.

5.4.2. Paralelismo y relleno (*padding*)

El segundo detalle relevante tiene que ver con el aprovechamiento de la potencia de cómputo. El uso de tarjetas gráficas (GPU) en el ámbito de la inteligencia artificial se justifica por su capacidad para procesar grandes volúmenes de datos en paralelo. Para explotar esta arquitectura, es estrictamente necesario que los datos se organicen en bloques o lotes (*batches*) de dimensiones homogéneas.

En nuestro caso, el procesamiento de lenguaje natural, nos enfrentamos al problema de que las frases tienen longitudes variables. En consecuencia, para que todas las frases de un lote tengan la misma dimensión y puedan ser procesadas simultáneamente por la GPU, se introduce el concepto de relleno (*padding*) el cual utiliza un *token* especial denominado <PAD>, perteneciente al vocabulario $\mathcal{V}$, para completar las frases más cortas hasta alcanzar una longitud máxima predefinida (en este proyecto, $N = 128$).

Por ejemplo, si nuestra longitud máxima es 6 y tenemos la frase “Me gusta el café”, la representación con relleno, considerando palabras como *tokens*, sería:

[Me, gusta, el, café, <PAD>, <PAD>]

Sin embargo, la introducción de *tokens* de relleno genera un inconveniente: el modelo podría aprender que el *token* <PAD> es una parte fundamental de la semántica de la frase debido a su alta frecuencia de aparición y podría dar a lugar a que el modelo aprendiese, única y exclusivamente, a predecir frases compuestas solo por el *token* <PAD>. Para evitar que el mecanismo de atención preste atención a este nuevo *token*, se aplica una máscara de relleno antes de la función *Softmax*.

Esta máscara de relleno no es más que añadir una matriz de máscara $\mathbf{M}$ al producto de las consultas y claves ($\mathbf{QK}^T$) y cuyos valores son $-\infty$ en todas las posiciones que corresponden a un *token* de relleno y 0 en caso contrario:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{QK}^T + \mathbf{M}}{\sqrt{D_i}}\right) \mathbf{V} \quad (5.7)$$

Dado que cualquier valor al que se le suma $-\infty$ da como resultado $-\infty$, al aplicar la función *Softmax* ($e^{-\infty} = 0$), las posiciones del relleno se anulan por completo. En consecuencia, el peso de atención sobre el relleno es cero, garantizando que estos *tokens* no afecten al cálculo de los pesos del resto de la secuencia.

En conclusión, al introducir esta máscara de relleno en la ecuación de atención, conseguimos que el modelo ignore los datos vacíos y se centre solo en la información útil de cada frase. Al estandarizar la longitud de las secuencias de esta manera, podemos procesar los datos en lotes aprovechando toda la potencia de cálculo de las tarjetas gráficas. Esto no solo mejora la calidad del aprendizaje, sino que acelera considerablemente el entrenamiento al explotar el paralelismo del hardware, permitiendo obtener resultados en menos tiempo.

Inglés (Original)	Español (Traducción)
Scarcely had the minister begun his speech when the protesters started chanting.	El Ministro apenas había iniciado su discurso cuando los manifestantes empezaron a cantar.
The intricate plot of the novel is underscored by a profound existential dread.	El plegado intrínseco de la novela es recortizado por una profunda hilo de reflexión.
The sweeping reforms implemented by the administration have drawn fierce criticism.	Las reformas dulces aplicadas por la administración han atraído críticas de gran resistencia.
It is of paramount importance that we address the underlying causes of inequality.	Es sumamente importante que nosotros abordemos las causas subyacentes de la inequidad.
To put it bluntly, their handling of the crisis was nothing short of disastrous.	Para darle una gran sensación de que la crisis no ha tenido ningún riesgo.
The candidate's ambiguous stance on environmental issues ultimately cost him the election.	El candidato es un ambiguo punto ambiente de que a finales de los aspectos medioambientales le costó la elección.
The findings of the longitudinal study corroborate the hypothesis posited earlier.	Ensayó sobre la evaluación de los análisis de longitudinal de la corroboró la hipótesis planteada anteriormente.
Given the current economic climate, investing in volatile markets is tantamount to gambling.	Habida cuenta del actual clima económico en los mercados, invertir en los mercados de libre tamaño es una contribución a la juego.
The artist's avant-garde exhibition challenges our preconceived notions of aesthetics.	La búsqueda de avant-garde presenta nuestras preconcebidas nociones de anesics.
What I find most baffling is their sheer reluctance to embrace technological advancements.	Yo encuentro que la mayoría de los inconvenientes es su ferviente renuencia para abrazar avances tecnológicos.

Tabla 5.8: Evaluación cualitativa de traducción - Nivel C1.

Inglés (Original)	Español (Traducción)
The sheer audacity of the embezzlement scheme left financial regulators utterly astounded.	La audición de la borda del esquema de encarnamiento dejó de los reguladores económicos importados de forma más agravante.
Her seminal thesis systematically dismantles the fallacies prevalent in pedagogical discourse.	Sus tesis seminales dismo sistemáticamente las consecuencias previas prevalecen en los discursos pedagógicos.
Amidst the cacophony of partisan bickering, his erudite arguments provided a beacon of rationality.	Amidst the Cacophony of partisan, sus argumentos eruditos dieron lugar al bacóna de racionalidad.
The treaty is fraught with legal ambiguities that could potentially precipitate international arbitration.	El tratado está a su vez consecuentes con ambigüedades jurídicas que podrían precipitar la arbitraje internacional.
The author masterfully weaves a tapestry of disparate narratives into a cohesive magnum opus.	El autor es más autorizado de aislamiento a un escudo de narrativas en un opio cohesivo.
It is a fundamental misconception to conflate correlation with causation in epidemiological studies.	Es una equivocación fundamental a la confusión de la correlación con causa de las causas en estudios epidemiológicos.
The draconian austerity measures precipitated a rapid deterioration of the socioeconomic fabric.	El dliticio de austeridad determinó la rápida deterioro del tejido socioeconómico.
His esoteric references, though intellectually stimulating, often alienated the layperson.	Sus referencias esotéricas, aunque intelectualmente estimulantes, muchas veces alienadas.
The insidious nature of the malware allowed it to circumvent cryptographic defenses with impunity.	La insidiosa naturaleza del Malware se le permite circuncidar las defensas racionales de muerte de forma asfixiante.
Bereft of all hope, the protagonist is thrust into a maelstrom of moral ambiguity.	A menudo de todos los esperaban que el protagonista se apresurara al de la ambigüedad moral.

Tabla 5.9: Evaluación cualitativa de traducción - Nivel C2.

Bibliografía

- [1] R. Aharoni, M. Johnson, and O. Firat. Massively multilingual neural machine translation. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, pages 3874–3884, 2019.
- [2] C. M. Bishop and H. Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024.
- [3] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models, 2020.
- [4] A. Karpathy. Let’s build GPT: from scratch, in code, spelled out. Video de YouTube, 2023. Disponible en YouTube.
- [5] A. Karpathy. Let’s build the GPT Tokenizer. Video de YouTube, 2024. Disponible en YouTube.
- [6] S. J. Prince. *Understanding Deep Learning*. The MIT Press, 2023.
- [7] S. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, Hoboken, NJ, 4th us edition, 2020.
- [8] J. Tiedemann. Parallel data, tools and interfaces in OPUS. In *Proceedings of the Eight International Conference on Language Resources and Evaluation (LREC’12)*, Istanbul, Turkey, may 2012. European Language Resources Association (ELRA).
- [9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Łukasz Kaiser, and I. Polosukhin. Attention is all you need, 2017.
- [10] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola. *Dive into Deep Learning*. Cambridge University Press, 2023. <https://D2L.ai>.

